Retrieval-Augmented Generation for Question Answering on Product Safety Standards

Shubhangi Phapale

Version: Template (0.0.0)



Department of Computer Science
Data Science Group (DICE)

Master Thesis

Retrieval-Augmented Generation for Question Answering on Product Safety Standards

A Thesis presented by

Shubhangi Phapale

to the
Faculty of Computer Science, Electrical Engineering and Mathematics
of
Paderborn University
in partial fulfillment of the requirements for the degree of
Master of Science

1. Reviewer Prof. Dr. Axel-Cyrille Ngonga Ngomo
Department of Computer Science
Paderborn University

2. Reviewer Dr. Michael Winkler
Forvia Hella

Supervisors Prof. Dr. Axel-Cyrille Ngonga Ngomo and
Dr. René Speck

Shubhangi Phapale
Retrieval-Augmented Generation for Question Answering on Product Safety Standards
Master Thesis,
Reviewers: Prof. Dr. Axel-Cyrille Ngonga Ngomo and Dr. Michael Winkler
Supervisors: Prof. Dr. Axel-Cyrille Ngonga Ngomo and Dr. René Speck

Paderborn University
Data Science Group (DICE)
Department of Computer Science
Faculty of Computer Science, Electrical Engineering and Mathematics
Warburger Str. 100
33098 and Paderborn

Abstract

Product safety standards are important in the automotive domain, but finding the right information in long PDF documents is difficult and time-consuming. These documents often contain technical terms, complex structure, and references across sections, which make it hard to answer questions. This thesis presents a retrieval-augmented generation system for question answering on automotive safety and compliance standards. The system retrieves relevant text passages from an indexed standards corpus and uses them as evidence for answer generation. A BM25 baseline is compared with a hybrid retrieval approach that combines lexical and dense retrieval. The effects of query expansion, multi-hop retrieval, and reranking are also examined. The approach is evaluated on a manually created 100-question dataset based on automotive standards PDFs and on a 300-question subset of the FRAMES benchmark. The results show that the hybrid approach gives the strongest overall performance on the standards dataset. Query expansion is especially helpful for BM25, while reranking further improves retrieval precision. Multi-hop retrieval can help in some cases, but it also increases runtime and may add irrelevant context. Overall, the results show that retrieval design strongly affects both the quality of evidence and the quality of the final answer.

Keywords: Retrieval-Augmented Generation, Large Language Models, Automotive Product Safety Standards, Vector Databases

Contents

1 Introduction **1**
- 1.1 Objective . . . 2
- 1.2 Research Questions . . . 3
- 1.3 Contributions . . . 3
- 1.4 Scope and Limitations . . . 4

2 Related Work **5**
- 2.1 RAG Architectures . . . 5
- 2.2 Retrieval Method Comparative Studies . . . 6
- 2.3 PDF and Structured Document RAG . . . 7
- 2.4 Evaluation Benchmarks and Datasets . . . 9
- 2.5 Domain-Specific RAG Applications and Engineering Challenges . . . 9
- 2.6 Research Gap and Thesis Focus . . . 10

3 Background **11**
- 3.1 Large Language Models . . . 11
  - 3.1.1 Token-based Generation and Context Windows . . . 11
  - 3.1.2 Hallucination and the Need for Grounding in Evidence . . . 12
- 3.2 Retrieval-Augmented Generation . . . 13
  - 3.2.1 Sparse Retrieval . . . 14
  - 3.2.2 Dense Retrieval . . . 15
  - 3.2.3 Hybrid Retrieval Approaches . . . 16
  - 3.2.4 Query Expansion . . . 16
  - 3.2.5 Multi-hop Retrieval . . . 16
- 3.3 Document Processing for Technical PDFs . . . 17
  - 3.3.1 PDF Processing and Text Extraction Challenges . . . 17
  - 3.3.2 Chunking Strategies . . . 18
- 3.4 Evidence Selection and Structured Content . . . 18
  - 3.4.1 Re-ranking . . . 18
  - 3.4.2 Table Extraction and Structured Content Handling . . . 19
- 3.5 Evaluation Methodologies for RAG Systems . . . 19

3.5.1 Retrieval Evaluation Metrics . . . 19
3.5.2 Reference-Free and Synthetic Evaluation . . . 20
3.6 RAG in Industrial and Technical Contexts . . . 20

4 Approach **23**
4.1 Data Collection . . . 23
4.1.1 Standards PDF Corpus . . . 23
4.1.2 100 QA Set Construction . . . 24
4.1.3 FRAMES Benchmark . . . 25
4.2 Document Processing . . . 26
4.2.1 The Challenge of Technical PDFs . . . 26
4.2.2 Parsing and Text Extraction . . . 27
4.2.3 Normalization Logic . . . 28
4.2.4 Chunking . . . 28
4.2.5 Indexing . . . 29
4.3 Retrieval Strategy . . . 29
4.3.1 Sparse Retrieval (BM25) . . . 30
4.3.2 Dense Retrieval (Embedding Similarity) . . . 30
4.3.3 Hybrid Retrieval . . . 32
4.3.4 Query Normalization and Expansion . . . 33
4.3.5 Multi-hop Retrieval . . . 33
4.3.6 Context Packing and Answer Generation . . . 37
4.4 Evaluation . . . 37
4.4.1 Retrieval Metrics . . . 37
4.4.2 Generation Metrics . . . 38

5 Implementation **39**
5.1 System Overview . . . 39
5.1.1 Technologies and Packages . . . 39
5.1.2 Runtime Configuration . . . 40
5.2 PDF Ingestion and Text Normalization . . . 41
5.2.1 PDF Loading . . . 41
5.2.2 Text Cleaning and Normalization . . . 41
5.3 Chunking and Indexing . . . 43
5.3.1 Chunking . . . 43
5.3.2 Indexing . . . 43
5.4 Retrieval Module Implementation . . . 44
5.4.1 Sparse Retrieval . . . 44
5.4.2 Dense Retrieval . . . 45

5.5 Hybrid Retrieval . . . 47
5.5.1 Reciprocal Rank Fusion (RRF) . . . 47
5.5.2 Cross-Encoder Reranking . . . 47
5.5.3 Domain-Specific Query Expansion . . . 47
5.5.4 Multi-hop Retrieval . . . 48
5.6 Context Packing Implementation . . . 49
5.7 Answer Generation . . . 49
5.8 Reproducibility and Practical Considerations . . . 49
5.9 Summary . . . 50

6 Evaluation **51**
6.1 Setup . . . 51
6.1.1 Comparative Analysis of Retrieval Strategies . . . 51
6.1.2 Datasets . . . 52
6.1.3 Models . . . 52
6.1.4 Metrics . . . 53
6.1.5 Evaluation Procedure . . . 53
6.1.6 Scope of Comparison . . . 53
6.2 Results . . . 54
6.2.1 PDF Dataset with GPT-4.1 . . . 54
6.2.2 FRAMES with GPT-4.1 . . . 59
6.2.3 PDF Dataset with GPT-OSS . . . 63
6.2.4 FRAMES with GPT-OSS . . . 68
6.2.5 Cross-Model Comparison . . . 72
6.3 Discussion . . . 74
6.3.1 Main Findings . . . 74
6.3.2 Effect of Query Expansion . . . 74
6.3.3 Effect of Multi-hop Retrieval . . . 74
6.3.4 Effect of Reranking . . . 75
6.3.5 100-QA and FRAMES Dataset Interpretation . . . 75
6.3.6 Runtime Trade-Offs . . . 77
6.3.7 Statistical Significance Analysis . . . 79
6.4 Evaluation Summary . . . 80
6.4.1 Answers to the Research Questions . . . 80

7 Summary **83**
7.1 Summary of the Thesis . . . 83
7.2 Limitations . . . 83
7.3 Future Work . . . 84

7.4 Conclusion 84

Bibliography **87**

A Appendix **109**
A.1 Dataset Links 109
A.2 Code 109
A.3 Prompt Templates 109
A.3.1 System Prompt 109
A.3.2 PDF Dataset Prompt 110
A.3.3 FRAMES Dataset Prompt 110
A.3.4 Multi-hop Judge Prompt 110

List of Figures

3.1 Retrieval-Augmented Generation (RAG) architecture [21] 13

4.1 Example of Complex Layout in Technical PDF 27
4.2 Baseline BM25 retrieval pipeline 31
4.3 Hybrid pipeline 34
4.4 Example abbreviation table from a standards document 35
4.5 Multi-hop Retrieval Workflow 36

List of Tables

4.1 Three illustrative examples from the 100-QA dataset 24
4.2 Sample FRAMES benchmark questions 25

5.1 Main Python packages used in the implementation. 39
5.2 Main runtime parameters used in the experiments 40

6.1 Retriever performance on the PDF dataset with GPT-4.1 54
6.2 Generation performance on the PDF dataset with GPT-4.1 54
6.3 Retriever performance with query expansion on the PDF dataset with GPT-4.1 55
6.4 Generation performance with query expansion on the PDF dataset with GPT-4.1 55
6.5 Retriever performance with multi-hop retrieval on the PDF dataset with GPT-4.1 57
6.6 Generation performance with multi-hop retrieval on the PDF dataset with GPT-4.1 57
6.7 Retriever performance with reranking on the PDF dataset with GPT-4.1 58
6.8 Generation performance with reranking on the PDF dataset with GPT-4.1 58
6.9 Retriever performance on FRAMES with GPT-4.1 59
6.10 Generation performance on FRAMES with GPT-4.1 59
6.11 Retriever performance with multi-hop retrieval on FRAMES with GPT-4.1 60
6.12 Generation performance with multi-hop retrieval on FRAMES with GPT-4.1 60
6.13 Retriever performance with reranking on FRAMES with GPT-4.1 62
6.14 Generation performance with reranking on FRAMES with GPT-4.1 . . . 62
6.15 Retriever performance on the PDF dataset with GPT-OSS 63
6.16 Generation performance on the PDF dataset with GPT-OSS 63
6.17 Retriever performance with query expansion on the PDF dataset with GPT-OSS 64
6.18 Generation performance with query expansion on the PDF dataset with GPT-OSS 64

6.19 Retriever performance with multi-hop retrieval on the PDF dataset with GPT-OSS 65
6.20 Generation performance with multi-hop retrieval on the PDF dataset with GPT-OSS 66
6.21 Retriever performance with reranking on the PDF dataset with GPT-OSS 67
6.22 Generation performance with reranking on the PDF dataset with GPT-OSS 67
6.23 Retriever performance on FRAMES with GPT-OSS 68
6.24 Generation performance on FRAMES with GPT-OSS 69
6.25 Retriever performance with multi-hop retrieval on FRAMES with GPT-OSS 69
6.26 Generation performance with multi-hop retrieval on FRAMES with GPT-OSS 70
6.27 Retriever performance with reranking on FRAMES with GPT-OSS . . . 71
6.28 Generation performance with reranking on FRAMES with GPT-OSS . . 71
6.29 Average retrieval time per question (seconds) for GPT-4.1. 78
6.30 Average retrieval time per question (seconds) for GPT-OSS. 78
6.31 Wilcoxon signed-rank test results for paired retriever F1 comparisons on the PDF dataset. 79
6.32 Wilcoxon signed-rank test results for paired retriever F1 comparisons on the FRAMES dataset. 79

List of Algorithms

5.1 Text normalization applied during PDF ingestion 42

5.2 Dense index construction 44

5.3 BM25 retrieval with optional query expansion and source filtering . . 45

5.4 Dense retrieval with optional query rewriting and FAISS search 46

List of Listings

5.1. Whoosh schema for BM25 sparse index. 43

1 Introduction

The automotive industry is going through a major transformation as modern vehicles include more embedded electronics, connectivity features, and advanced software functions [48, 49]. Due to the increasing use of software-controlled features and connected services, product development and certification must meet strict standards and regulations [14, 54]. Furthermore, manufacturers have to comply with regional regulatory frameworks such as Federal Motor Vehicle Safety Standards (FMVSS) [1] in the United States, which have their own requirements, and assessment procedures [38]. Modern automotive products have to meet a set of requirements across multiple technical domains and regulatory environments [14, 38, 54]. Generally, this reflects a broader document-understanding challenge in which users must answer questions over long, structured, and often heterogeneous documents rather than short, self-contained text passages [65].

Engineers and compliance specialists must identify, understand, and verify relevant clauses within lengthy, complex, and terminology-heavy documents. Manual review is time-consuming and error-prone, particularly when key conditions, exceptions, or cross-references appear in multiple sections or documents [14, 38]. While recent research has examined automated tools for standards analysis and question answering, reliable evidence-grounded support remains limited. This challenge is especially pronounced in high-stakes compliance settings where unsupported or unverifiable answers are not acceptable [1, 27]. As a result, retrieval quality is critical in document question answering, since the evidence retrieved directly influences the generated answer. Poor retrieval can reduce both performance and factual accuracy [42].

Retrieval-augmented generation (RAG) offers a promising approach to addressing this problem. Especially for question answering over long document collections, where relevant knowledge must be retrieved from external sources rather than concluded solely from model parameters [42]. RAG provides answers based on retrieved evidence passages, which makes it easier to trace and check the answers against reliable sources [37, 97]. However, retrieval over long and structured documents remains challenging, particularly in domain-specific contexts such as

[1] https://www.nhtsa.gov/laws-regulations

automotive standards PDFs [14, 65]. Locating relevant evidence is complicated by paraphrasing, domain-specific abbreviations, complex document structures, and dependencies across sections or documents. RAG remains useful even though recent language models can process longer inputs [42]. In standards-based question answering, documents are often long, complex, and spread across many files [1, 14]. This is why, it's not a good idea to put all documents directly into the prompt, especially in bigger real-world settings. Even if this is possible, it can add unnecessary information and increase cost. RAG is still useful because it helps the system focus on the most relevant passages and produce answers that can be checked against the source documents [42, 97]. This thesis investigates retrieval-augmented question answering for automotive standards PDFs, focusing on the impact of retrieval design on retrieval quality and final answer quality.

1.1 Objective

The aim of this thesis is to develop and evaluate a retrieval-augmented question answering system for automotive safety and compliance standards provided as PDF documents, which serve as the primary application domain. The proposed system is designed to enable engineers to ask natural-language questions and receive answers supported by retrieved evidence from an indexed standards corpus.

The research focuses on the impact of retrieval strategies on retrieval quality, evidence quality, and downstream answer quality. To study this, the thesis compares a keyword-based BM25 baseline with a hybrid retrieval pipeline that combines lexical and dense retrieval through rank fusion. In addition, the work evaluates the effects of query expansion, multi-hop retrieval, and reranking to determine which retrieval components are beneficial in the standards domain and whether these effects remain robust in a more challenging cross-domain setting.

The approach is evaluated on a domain-specific 100-question dataset constructed from automotive standards PDFs and on a 300-question subset of the FRAMES [39] benchmark. This design makes it possible to study both in-domain behavior and cross-domain generalization, and to identify failure patterns in cases where retrieval improvements do not necessarily lead to better final answers. The retrieval pipeline is evaluated with two answer-generation models, GPT-4.1 and GPT-OSS, to see whether the same retrieval patterns hold across different model types.

1.2 Research Questions

The thesis is guided by the following research questions:

RQ1: How does a baseline lexical RAG system compare with a hybrid lexical semantic RAG system in terms of retrieval quality and answer quality for automotive standards question answering?

RQ2: To what extent do retrieval enhancements such as query expansion, multi-hop retrieval, and reranking improve evidence quality and final answer quality in automotive standards question answering?

RQ3: How do the baseline and hybrid retrieval strategies transfer from domain-specific automotive standards questions to a cross-domain benchmark such as FRAMES?

RQ4: How consistent are the observed retrieval and answer-quality trends across the two evaluated language models, GPT-4.1 and GPT-OSS?

1.3 Contributions

This thesis makes the following contributions:

- It develops a retrieval-augmented question answering system for automotive standards PDFs.
- It implements and compares a BM25-based baseline with a hybrid retrieval pipeline that uses both lexical and dense retrieval.
- It evaluates the effects of retrieval enhancements, including query expansion, multi-hop retrieval, and reranking.
- It provides an empirical evaluation on an automotive standards dataset and the FRAMES benchmark.
- It compares the performance of the approach using OpenAI models and open-source language models.

1.4 Scope and Limitations

This thesis focuses on comparing retrieval strategies for question answering over an indexed corpus of automotive standards and regulatory PDF documents. The main focus is on retrieval quality, evidence quality, and answer quality.

The evaluation uses two datasets. The first is a domain-specific 100-question dataset created from 20 publicly available automotive standards PDFs, where most questions can be answered from a single source document.

The second dataset is the FRAMES benchmark. Only the first 300 questions were used in this thesis to maintain computational feasibility while ensuring a sufficiently large and diverse sample.

In addition, the evaluation is limited to the selected datasets, retrieval configurations, and language models considered in this thesis. The results show how well the specific retrieval methods studied in this thesis perform. They only provide evidence for these particular approaches and do not cover all possible ways to retrieve or generate answers for standards-based question answering.

Related Work

2

2.1 RAG Architectures

Previous studies show that external knowledge can improve the factual accuracy and domain usefulness compared with relying only on internal model parameters [18, 42, 82]. Early RAG formalizes this idea by pairing a parametric sequence-to-sequence generator with a non-parametric retriever over an external corpus, enabling generation conditioned on retrieved passages [18, 42]. Such generator architectures commonly include models such as BART [41]. Related work extends this standard pipeline in several ways, including retrieval-augmented pretraining, which learns retrieval during pretraining for knowledge-intensive tasks [22]. It also includes retrieval-augmented models such as Atlas, which improve few-shot performance on knowledge-intensive tasks through retrieved evidence [29].

After these early designs, later studies propose distinct RAG architectures. Recent surveys categorize RAG research into main architectural perspectives, including retrieval, generation, and augmentation [8, 18]. Recent approaches introduce adaptive or iterative retrieval. For example, Self-RAG retrieves information on demand and critiques its own generations through self-reflection to improve generation quality and factuality [2].

Corrective RAG (CRAG) addresses low-quality retrieval by using a lightweight retrieval evaluator to detect poor evidence [85]. It then triggers corrective actions, such as web search and document-level refinement, to get higher-quality supporting documents. Beyond text-corpus retrieval, graph-based RAG approaches retrieve structured graph evidence, including nodes, paths, or subgraphs. These methods enable architectures that explicitly leverage relational structure for multi-hop reasoning and evidence grounding [23, 26, 67]. In tool-based settings, ReAct integrates step-by-step reasoning with actions that query external sources, enabling iterative information gathering and multi step processes [87].

These architectures involve trade-offs among simplicity, robustness, controllability, and implementation complexity [8]. Standard RAG remains relatively simple and effective when the corpus is reliable and evidence is primarily textual [18]. However,

it does not directly address scenarios where retrieval is unnecessary or where retrieved evidence is weak [2, 70, 85]. To address this limitation, SELF-RAG can skip retrieval when not needed and critique its own outputs. However, this approach adds more complexity to the computational and control due to its reflection mechanism [2]. CRAG further identifies retrieval failures and applies corrective strategies, including optional web search, thereby improving robustness when retrieval quality is inconsistent [85]. However, the additional components, including the retrieval evaluator, refinement module, and external search, increase system complexity and may be harder to deploy in restricted environments. In contrast to text-corpus approaches, graph-based RAG methods are particularly effective for tasks that depend on entities, relations, or global corpus structure. These methods organize evidence as graphs rather than relying only on standard vector retrieval [16, 23]. Beyond these core architectural designs, agentic and tool-based approaches such as ReAct are effective for tasks that require iterative querying and interaction with external environments [87]; however, they also increase system complexity and dependence on external tools or retrieval infrastructure [8, 18]. For a controlled retrieval comparison, however, such approaches introduce additional decision stages that make it harder to isolate the effect of the retriever itself. Therefore, this thesis adopts a simpler controlled retrieval pipeline rather than an adaptive or agentic system.

2.2 Retrieval Method Comparative Studies

Since RAG architectures rely on the top-k retrieved documents or chunks as evidence, comparative studies of retrieval methods are essential. These studies indicate which retrievers provide stronger retrieval effectiveness for downstream generation. Large-scale benchmarks such as BEIR evaluate lexical, dense, late-interaction, and re-ranking retrieval models in a zero-shot setting across diverse datasets [76]. BEIR reports that BM25 remains a strong lexical baseline, while late-interaction and re-ranking methods often achieve higher zero-shot effectiveness, although they require more computation. In open-domain question answering, dense retrieval shows a big improvements over BM25 in top-k retrieval accuracy. This shows how important semantic matching is when lexical overlap is minimal [33].

Kuzi et al. [40] demonstrate that hybrid retrieval systems, which combine BM25 with semantic retrieval, enhance recall by capturing both exact-match and semantically relevant documents. Empirical analysis further shows that hybrid retrieval is generally robust across queries: it improves performance for many queries and

degrades only a smaller subset [40]. Bruch et al. analyze hybrid fusion strategies and show that weighted score-level fusion can be effective when a small validation set is available for tuning [10]. Beyond score-level fusion, modern RAG pipelines commonly incorporate re-ranking components that refine retrieval results without modifying the underlying index, enabling more precise relevance estimation before generation [5, 89].

These comparative findings motivate the design choices addressed in this thesis. Here, BM25 provides a strong and interpretable lexical baseline. Hybrid retrieval is also relevant, as previous studies indicate that integrating lexical and semantic signals can enhance retrieval performance by utilizing their complementary strengths [10, 40]. Re-ranking is included as a final-stage refinement step that can improve ranking quality [5, 89]. Prior work also studies query reformulation and query expansion, which are used to reduce vocabulary mismatch by adding semantically related or contextually useful terms to the original query. Such methods have long been used in information retrieval to improve recall when the user query does not match the terminology used in the documents [4]. In terminology-heavy domains, query reformulation is especially relevant because abbreviations, alternate phrasings, and domain-specific wording can otherwise reduce lexical matching quality.

Related work also shows that some question answering tasks benefit from iterative or multi-hop retrieval, where retrieval is repeated after intermediate reasoning or query reformulation rather than performed only once. Such approaches are particularly useful when the evidence needed to answer a question is distributed across multiple passages or documents. For example, IRCoT combines retrieval with intermediate reasoning steps, while earlier iterative retrieval methods repeatedly reformulate or extend the search process to gather missing evidence for multi-step questions [60, 75, 78]. Overall, prior work suggests that more adaptive or agentic RAG variants may improve robustness. However, they also add complexity, which makes simpler hybrid retrieval pipelines more suitable for controlled experimental comparison.

2.3 PDF and Structured Document RAG

In PDF-based RAG systems, retrieval quality depends strongly on document parsing and segmentation, because extraction errors and incorrect chunk boundaries can affect indexing and downstream generation [56, 65]. Zhu and Cole [96] show that common PDF layout extraction tools, such as PDFMiner, can recover the reading

order of text blocks, but provide very limited semantic information and do not identify the logical role of each block. To address these limitations, PDFDataExtractor reconstructs logical article structure and outputs structured metadata (e.g., title, sections, references) alongside extracted text [96]. In particular, an experience report on building RAG systems from PDFs [34] shows that design choices in document parsing, text cleaning, and chunking directly affect retrieval quality. These preprocessing decisions strongly influence overall performance by determining what information can be retrieved and how clearly it is presented to the model. However, prior work reports that naive fixed-length or sliding-window chunking can fail to preserve semantic coherence and structural relationships in complex technical PDFs [77]. While traditional approaches rely on fixed-size chunking [34], recent work has explored structure-aware alternatives that incorporate document layout and logical organization to better preserve semantic coherence [77]. Recent work proposes structure-aware retrievers that leverage document structure during contrastive training [28]. These methods incorporate element-level alignment to capture fine-grained semantic relationships in long structured documents, improving retrieval performance on structured-document benchmarks.

Beyond plain text, tables pose distinct challenges for RAG systems. Recent work [69] therefore treats table extraction and representation as a dedicated step, aiming to recover table structure and enable downstream use of structure over table content. Table-centric question answering and reasoning approaches show that effective use of tables often requires explicit modeling of table structure, such as rows, columns, and headers [12, 88, 91, 92]. This motivates table-aware retrieval and reasoning components in RAG pipelines. The results also show that table retrieval performance is affected by metadata and table context, such as titles and column headers, highlighting the need for table-aware representations and indexing strategies [31]. Models such as LayoutLM [83] take both text and page layout into account, and datasets like DocVQA [52] evaluate question answering directly on document images. These studies show that document understanding often depends not only on text content but also on layout and visual structure, which text-only pipelines may only partially preserve. For this reason, recent work explores multi-modal document understanding for RAG, combining extracted text with layout and visual cues to improve evidence preservation in complex PDFs [56, 77].

This review motivates a text-based PDF pipeline as a controlled basis for retrieval comparison. Rather than introducing multimodal parsers or specialized table-reasoning modules, the thesis focuses on how retrieval strategy changes performance when preprocessing is kept fixed.

2.4 Evaluation Benchmarks and Datasets

Prior work evaluates knowledge-grounded generation systems using benchmarks that assess both answer quality and evidence attribution [63]. KILT [58] provides a unified benchmark for knowledge-intensive language tasks by aligning diverse datasets to a single Wikipedia snapshot and supplying provenance annotations, enabling standardized evaluation of both retrieval and generation across tasks. FRAMES [39] introduces a RAG-oriented evaluation framework that jointly assesses answer correctness, retrieval quality, and reasoning within an end-to-end pipeline. Beyond the benchmarks themselves, evaluation design choices also matter. Prior evaluation surveys note that the choice of top-k directly trades off retrieval recall against context noise, and some studies analyze performance across different k values to better understand system behavior under varying context budgets [8, 89]. In addition, engineering analyses report that many end-to-end failures originate from retrieval and context-selection errors [5]. This suggests that evaluation should consider both retrieval quality and final answer quality rather than relying only on end-to-end outputs. These benchmark findings motivate evaluating both retrieval-level and answer-level behavior rather than relying only on final generated answers [39, 58]. In this thesis, experiments on a domain-specific standards dataset and the FRAMES benchmark are used to study both in-domain performance and cross-domain generalization.

2.5 Domain-Specific RAG Applications and Engineering Challenges

RAG has been applied in regulatory compliance systems [37], financial document analysis [93], and academic literature search tools [3], showing its applicability across knowledge-intensive domains. Recent case studies in the automotive and manufacturing context highlight practical design choices such as local model deployment, PDF-specific preprocessing, and retrieval tuning to improve answer relevance while maintaining privacy constraints [45, 50]. Prior work describes the practical challenges and implementation difficulties faced in real-world RAG deployments [34]. In particular, chunking decisions play a central role in retrieval effectiveness and must be calibrated as part of the overall system design, because end-to-end quality depends strongly on retrieval and context construction [5, 34].

2.6 Research Gap and Thesis Focus

Despite the diversity of RAG techniques and documented engineering challenges, there is limited systematic evaluation of how retrieval strategy and reranking affect PDF-based RAG workflows under controlled preprocessing and fixed context-budget constraints.

This thesis addresses this gap by studying how retrieval design affects PDF-based RAG under controlled preprocessing conditions, with particular attention to retrieval quality, answer quality, and the effect of optional retrieval refinements such as query expansion, iterative multi-hop retrieval, and reranking. More complex alternatives, such as agentic control, web search, graph construction, or adaptive retrieval policies, are outside the scope of this thesis because they would introduce additional variables and reduce experimental control over the retrieval comparison.

3 Background

3.1 Large Language Models

Large Language Models (LLMs) are neural language models that can understand and generate text across a wide range of tasks [25, 53]. LLMs are neural network architectures based on the Transformer model, often containing billions of parameters trained on extensive text corpora [53, 79]. LLMs perform well on a range of language tasks, including reasoning [80], question answering [9], summarization [71], and instruction following [57]. In practice, an LLM generates text token by token, predicting each next token based on the input and the tokens it has already produced. This enables flexible generation but it also means that the output depends on the provided context and the model's learned parameters [84]. The following subsections explain token-based generation, context-length limitations, and the need for evidence grounding.

3.1.1 Token-based Generation and Context Windows

LLMs generate text using a token-based process, where text is represented as a sequence of discrete units called tokens. Generation proceeds sequentially, with each token produced one after another. This process is controlled by a probability distribution in the vocabulary. The model uses the context of previous token to figure out how likely each possible next token is [53]. In pre-training, the language model is optimized to predict the next token in a sequence using a likelihood-based objective. Through this optimization process over large text documents, the model learns general patterns present in the training text [9, 53]. In practice, language models operate with a fixed maximum context that limits the amount of text that can be processed at once [9, 53]. To mitigate fixed context-window limits, researchers have explored conditioning generation on additional context beyond the immediate prompt, for example by supplying summaries or selected supporting passages alongside the query [42, 43]. Recent LLMs support longer input lengths, but their performance degrades as the input grows longer, and important information may be overlooked in very long texts [46]. This limitation motivates

approaches that provide the model with only relevant text, rather than the full document, during answer generation [42]. Beyond context limitations, LLMs face a fundamental reliability problem: hallucination, which will be explained in the next section [27].

3.1.2 Hallucination and the Need for Grounding in Evidence

A major reliability issue in LLMs is hallucination: the generation of responses that sound convincing but are factually incorrect. This is often influenced by biases in training data and by next-token prediction, which can lead the model to produce plausible continuations even when evidence is missing or uncertain [44]. Prior studies commonly distinguish between two categories of hallucination: intrinsic hallucinations, where the model contradicts the given context, and extrinsic hallucinations, where it introduces unsupported information beyond the evidence. This distinction is critical when evaluating RAG systems, as it clarifies whether failures stem from context misuse or from the model's reliance on parametric knowledge [27].

Since language models are trained to predict likely tokens rather than verify facts, they may generate fluent but incorrect statements when relevant information is missing from the provided context [66, 84]. Additionally, as model parameters are set after training, LLMs lack access to real-time information and must rely solely on their training data, causing limitations in field knowledge-intensive domains where current and precise information is crucial [44].

Even when relevant context is provided in the prompt, LLMs may still rely on biased or outdated parametric knowledge obtained during training, resulting in intrinsic hallucinations where the generated output contradicts the offered evidence [66, 84]. To overcome this dependency on parametric knowledge, external knowledge sources can be connected with LLMs to provide up-to-date and grounded context, thereby reducing hallucination and improving factual consistency [42, 73].

These limitations show why LLM outputs need to be grounded in reliable external sources. Grounding helps ensure that generated answers are not only fluent, but also factually correct and traceable to trusted evidence [42, 73]. In domains where accuracy is critical, such as safety and regulatory compliance, reliance on unaided model outputs is especially problematic [42, 54]. Overall, LLMs are powerful generators, but hallucinations and context limitations make their unaided outputs unreliable. These limitations motivate RAG, where relevant evidence is retrieved and provided as context to improve factuality and traceability.

3.2 Retrieval-Augmented Generation

To address these challenges, RAG integrates an information retrieval component with a generative model. RAG conditions LLM generation on retrieved document passages, allowing the model to use both retrieved evidence and its learned parameters when generating an answer [42]. At inference time, a user query is matched against a corpus of indexed documents through a retriever component, which returns the top-k most relevant passages. These passages are then merged with the original query and provided to the LLM, which generates an answer informed by both the retrieved evidence and its learned parameters [42]. The performance of a RAG system depends on retrieval quality. If the retriever fails to identify relevant documents, the generator lacks sufficient context to produce accurate and reliable responses [5]. As a result, the design and evaluation of effective retrieval strategies play an important role in determining the robustness of RAG systems, particularly when applied to large and structurally complex document collections [5, 18]. This chapter introduces the general RAG pipeline and its core technical components at a conceptual level; more specialized system variants and literature comparisons are discussed in Chapter 2.

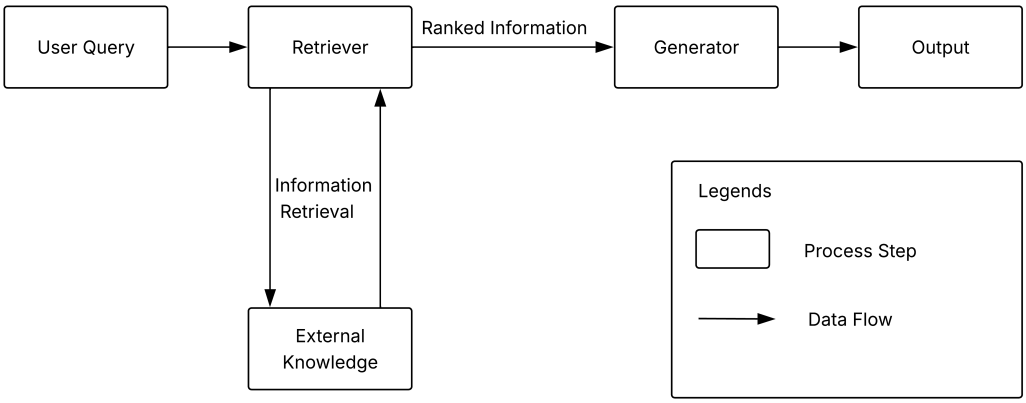


Figure 3.1.: Retrieval-Augmented Generation (RAG) architecture [21]

A RAG system integrates retrieval and generation components that operate on external data sources. The process begins with a knowledge base, which contains domain-relevant information organized for efficient access. This knowledge undergoes preprocessing to identify and segment coherent chunks, reduce noise, and prepare data for indexing [42]. Then chunks are indexed to enable efficient retrieval at query time [32, 42]. In dense retrieval settings [62], these chunk representations are typically stored in a vector index (or vector store) [15]. It maintains embedding vectors together with references to the original text units so that similar chunks

can be retrieved efficiently at query time. Instead of checking every single chunk embedding, vector indexes use nearest-neighbor or approximate nearest-neighbor search. This finds the vectors that are most similar to the query embedding based on a chosen similarity measure [51]. This makes dense retrieval practical for large collections, where exact comparison against every stored vector would otherwise require a lot of time and computing power [15, 51].

During query processing, the retriever searches the index to identify the top-k relevant chunks [42]. These retrieved chunks are combined into a context window and provided to the LLM, which generates an answer grounded in the retrieved evidence. This approach allows responses to be grounded in external knowledge sources instead of relying only on what the model has learned internally [42].

The retriever component is critical to RAG effectiveness. Different retrieval methods balance lexical precision against semantic flexibility, with distinct trade-offs. The following subsections describe three main retrieval : sparse (lexical), dense (semantic), and hybrid approaches.

3.2.1 Sparse Retrieval

Sparse retrieval methods determine relevance by matching words between the query and the documents. BM25 is a widely adopted scoring-based approach that scores documents d for a query q by using term frequency, inverse document frequency, and normalization based on document-length [64]:

$$\text{BM25}(d,q) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i,d) \cdot (k_1+1)}{f(q_i,d) + k_1(1 - b + b\frac{|d|}{L_{avg}})}$$

- q: The query, consisting of a set of search terms
- d: A document in the collection
- q_i: The i-th term in the query
- $\sum_i$: Summation over all query terms q_i
- BM25(d,q): Relevance score of document d with respect to query q
- IDF(q_i): Inverse document frequency of q_i, reflecting its importance across the document collection
- $f(q_i,d)$: Term frequency of q_i in document d
- k_1: Parameter controlling term frequency saturation
- b: Parameter controlling document length normalization
- $|d|$: Length of document d

- L_{avg}: Average document length across the collection

Because BM25 relies on exact term overlap, it is particularly effective for identifier- and keyword-heavy queries [40, 68]. In safety standards and regulatory documents, and technical terms are often highly specific and must be matched exactly, so lexical retrieval methods such as BM25 can achieve high precision [40, 50]. However, strict lexical matching can miss relevant passages when the same concept is expressed using different terminology, motivating dense retrieval methods that incorporate semantic similarity [40].

3.2.2 Dense Retrieval

While sparse retrieval excels at exact-match queries, it struggles with semantic reformulations. Dense retrieval addresses the lexical limitations of sparse retrieval by representing queries and chunks as dense vector embeddings, so it can find related content even when there is little or no token overlap [18, 82]. Many dense retrievers use a bi-encoder architecture that encodes queries and chunks independently and are trained using a contrastive ranking objective incorporating in-batch and hard negative examples to separate relevant evidence from non-relevant passages [33]. In dense retrieval models, relevance is computed using vector similarity between the query embedding and the stored chunk embeddings:

$$s(q, d) = \mathrm{sim}(\mathbf{e}_q, \mathbf{e}_d),$$

where $\mathbf{e}_q$ and $\mathbf{e}_d$ are the query and chunk embeddings, and sim is typically cosine similarity or inner product [33]. Compared to BM25, embedding-based retrieval can match semantically related passages even when lexical overlap is low [18]. In practice, these embeddings are searched through a vector index, which supports efficient similarity search over large embedding collections[15].

A key limitation of bi-encoder retrieval models is that relevance is compressed into a single similarity score between independently encoded queries and chunks [33]. This means the model cannot capture precise token-level interactions between query terms and passage content. Instead it produces fixed-size dense vectors that summarize relevance signals in a single, global representation. This forces reliance on semantic similarity at the embedding level, which can miss exact word matches and subtle patterns that token-level may better capture [33].

3.2.3 Hybrid Retrieval Approaches

Hybrid retrieval combines results from sparse and dense retrievers using a fusion step to produce a single ranked list. This is often beneficial because lexical and semantic signals are complementary, so fusion can improve evidence coverage compared to using a single retriever [10]. One common rank-based method is Reciprocal Rank Fusion (RRF), which combines multiple ranked lists by assigning higher weight to documents that appear near the top positions in those lists [13]. Alternatively, hybrid retrieval systems can combine sparse and dense methods by weighting both keyword relevance and vector similarity scores to produce a final ranking [10, 74]. Because these scores are typically on different scales, score fusion commonly applies normalization before weighting and aggregation [10].

3.2.4 Query Expansion

Query expansion improves retrieval by rewriting the original user query into a richer form that is more likely to match relevant evidence in the document collection. This can be done by adding synonyms, related terms, abbreviations, or domain-specific variants of key expressions [7, 72]. The main goal is to reduce the difference between how the query is phrased and how the same ideas are expressed in the source documents [7, 20]. In technical and standards-oriented corpora, this is especially relevant because the same concept may appear in both abbreviated and fully expanded forms. Query expansion is a technique that helps retrieval by improving evidence recall, though expanding too much can add noise and lower precision.

3.2.5 Multi-hop Retrieval

Some questions cannot be answered from a single retrieved passage and instead require combining evidence from multiple pieces of text [86]. Multi-hop retrieval refers to retrieval strategies that gather evidence iteratively across more than one step, rather than relying on a single retrieval round [19]. In a typical multi-hop setup, the system first retrieves an initial set of passages, then uses the intermediate evidence to reformulate or refine the search for additional supporting information. Conceptually, this allows the retrieval process to follow an evidence chain across multiple chunks or documents. While this can help answer complex questions, but it

may also increase runtime and drift from the original topic if later retrieval brings in unrelated information.

3.3 Document Processing for Technical PDFs

3.3.1 PDF Processing and Text Extraction Challenges

Even with improvements in sparse, dense, hybrid retrieval, and re-ranking methods, many RAG pipelines are still primarily designed around chunked textual inputs and standard sequential retrieval workflows [82]. Plain text documents are linear and easy to process, whereas PDFs are created for visual presentation and often contain complex layouts such as multi-column text, tables, and embedded figures [96]. These structural qualities create issues for text extraction and retrieval pipelines, as PDFs are designed for displaying content visually rather than machine reading, therefore converting them into clean text often loses information. Multi-column layouts can disrupt reading order during PDF parsing [90]. In addition, repeated artifacts such as headers and footers add noise to extracted text, and line-level formatting (e.g., line breaks, hyphenation, and column layouts) can fragment terms, making tokenization and retrieval less reliable [6, 96].

Recent document-parsing studies show that PDF ingestion performance is influenced by document structure and layout complexity [59, 96]. Large-scale layout benchmarks such as DocLayNet [59] highlight the diversity and structural complexity of real-world documents, reinforcing the difficulty of robust PDF parsing. Together, these design choices show that effective PDF processing must handle significant layout differences to ensure text can be accurately chunked and retrieved.

In addition to layout differences, PDFs usually do not provide reliable semantic labels for extracted elements such as headings or captions, which complicates downstream indexing and document structuring [96]. Even with block-level metadata, PDF-to-text output can remain noisy and structurally incomplete. In practice, retrieval performance in PDF-based RAG systems depends heavily on the quality of PDF-to-text conversion, because extraction errors can disrupt semantic coherence. To address these challenges, robust document processing requires cleaning, normalization of extracted text, and segmentation into semantically meaningful chunks before embedding and indexing [34].

3.3.2 Chunking Strategies

In RAG systems, documents are split into smaller pieces before retrieval. The size of these pieces matters: very small chunks can miss important context, while very large chunks often include irrelevant information. This creates a trade-off between coverage and precision, which has motivated recent work on using flexible or mixed chunk sizes instead of a single fixed setting [95]. Beyond chunk size, chunk boundaries also affect retrieval quality, since splitting in the middle of meaningful units can result in incomplete evidence [34, 61, 95]. This issue is especially relevant for technical PDFs, where extraction noise and layout artifacts may fragment coherent content. As a result, chunking is not only a preprocessing step, but a key design choice that shapes evidence quality for both retrieval and generation [34, 95].

3.4 Evidence Selection and Structured Content

3.4.1 Re-ranking

The challenge of selecting the most relevant evidence from a large pool of candidate chunks persists even when chunking quality is optimized. Re-ranking is typically implemented as a two-stage ranking setup, where an initial retriever returns a small candidate set and a second model re-scores these candidates using richer query-document analysis [35, 55]. This design aligns well with RAG pipelines, which rely on high-quality top-k evidence. As shown in ColBERT [35], re-ranking can be applied only to the retrieved candidate set rather than the full collection, and prior work reports that improved evidence selection can enhance downstream generation quality [42]. In information retrieval, this second-stage refinement is commonly formulated as a *learning-to-rank* problem, where the models learn to order candidate documents according to estimated relevance [11, 47]. Beyond full cross-encoder rerankers, late-interaction models such as ColBERT provide a compromise between efficiency and expressive matching [35]. This design enables efficient top-k retrieval over large candidate sets using approximate nearest neighbour indexing. Conceptually, approximate nearest-neighbor indexing trades a small amount of accuracy for much faster retrieval. This makes it useful for large-scale systems and explains why it is widely used in dense and late-interaction retrieval systems [35, 51]. However, re-ranking alone does not resolve failures caused by non-linear or structured evidence (e.g., tables). These tasks require maintaining row-column structure rather than simply improving text-chunk ordering [24, 88].

3.4.2 Table Extraction and Structured Content Handling

Tables represent organized and relational data, but extracting them from PDFs is challenging because correct interpretation depends on preserving table structure, including rows, columns, and cell relationships [69]. Moreover, tables are flattened into linear text sequences for downstream processing, which makes explicit row-column embeddings necessary to preserve structural information [24]. Beyond text and tables, many technical PDFs include figures and other visual elements that are not fully represented in plain-text extraction. Some documents contain important information in the text, and in layout or visual structure. This shows that text-only RAG pipelines can miss information that appears in figures, diagrams, or scanned pages [52, 83].

3.5 Evaluation Methodologies for RAG Systems

Evaluating a RAG system requires measuring the retriever's effectiveness in identifying relevant evidence and the generator's capacity to produce accurate responses supported by that evidence.

3.5.1 Retrieval Evaluation Metrics

For retrieval, common metrics include precision@k and recall@k, which measure how many of the top-k retrieved chunks are relevant and how many relevant chunks are successfully retrieved [8, 89]. This can also be summarized with an F1-score, which balances precision and recall [8]. Ranking-sensitive metrics such as MRR@k and nDCG@k are commonly used to evaluate retrieval quality, as they emphasize the position of relevant evidence in the ranked list [8, 30, 89]. This is particularly important in RAG settings, where only the top-k retrieved chunks are typically provided as context to the generator [8, 89]. These measures are computed with respect to a reference set of relevant chunks or documents for each query, against which retrieved items are labeled as relevant or non-relevant [89].

For generation, recent work also uses automated approaches, including LLM-based judging, to assess properties such as answer correctness, relevance, and faithfulness [8, 89, 94].

3.5.2 Reference-Free and Synthetic Evaluation

When labeled test data is not available, practitioners often evaluate RAG systems using synthetic question-answer generation or by piloting the system under minimal testing conditions [5]. RAGAS [17] is one such framework that supports reference-free evaluation with automated metrics for context quality and how well the content is supported by evidence, which is useful when large-scale manual labeling is not possible. This evaluation gap highlights a critical mismatch: strong retrieval performance does not guarantee correct answers if the retrieved chunks are incomplete, noisy, or poorly formatted for the generator [5, 17, 89]. In industrial and standards-related settings, the documents are often sensitive, highly structured, and contain a mixture of text, tables, and illustrations, which makes both reliable retrieval and secure deployment key requirements [50, 65]. Prior work shows that RAG can reduce knowledge limitations of LLMs by grounding responses in retrieved evidence, but end-to-end quality depends on strongly retrieval and context construction [97]. For PDF-based RAG, this issue is often greater because extraction noise, layout complexity, and table structure can degrade chunk quality and ranking, and make retrieval and ranking less reliable [34].

3.6 RAG in Industrial and Technical Contexts

Industrial organizations increasingly rely on large collections of internal documents such as standards, safety cases, technical manuals, and cybersecurity policies. In industrial settings, RAG is increasingly applied to large collections of technical PDF documents. In these settings, answers need to be both correct and traceable to a reliable source because mistakes can lead to compliance issues or safety risks [36, 38, 50]. At the same time, knowledge is continuously updated (e.g., new requirements, revised procedures), which makes parametric model knowledge difficult to maintain over time [42].

RAG is therefore relevant in industrial deployments because it can ground responses in retrieved passages from controlled document repositories, which improves accuracy and enabling traceability to the original sources [42, 73]. However, using RAG in industry brings extra requirements beyond open-domain QA. Many document collections are proprietary or contain sensitive information, making on-premise or privacy focused deployments necessary instead of relying solely on the cloud [45, 50].

A practical challenge is that industrial knowledge is often stored as complex PDFs containing tables, figures, and non-linear layouts. As a result, end-to-end quality depends not only on the retriever and generator, but also on ingestion choices (parsing, chunking, and metadata), and on careful evaluation of failure modes across the pipeline [5, 34]. These characteristics show that robust industrial PDF-RAG depends on both retrieval strategy and document processing quality, rather than on model choice alone.

4 Approach

This chapter describes the proposed approach and the design choices of the end-to-end RAG pipeline. RAG has been widely applied to question answering over PDF documents across domains [34, 37, 45]. This thesis studies the impact of the retrieval strategy on evidence quality and answer quality under a controlled setting. Two pipelines are compared under identical preprocessing and generation conditions. The baseline pipeline uses sparse retrieval (BM25) to rank and select the top-k chunks for context construction. The hybrid pipeline, illustrated in Figure 4.3, combines sparse and dense embedding-based retrieval. It merges both ranked lists using Reciprocal Rank Fusion (RRF) [13]. Optionally, a cross-encoder reranker [55] refines the fused candidate set before the final context is constructed. To improve robustness to standards-specific terminology (e.g., abbreviations, units), the approach includes an optional query normalization and expansion module. For sparse retrieval, the expanded query retains lexical matchability by including both abbreviations and their long forms. For dense retrieval, the query is rewritten into a compact, embedding-friendly form to avoid Boolean operators and excessive query noise. In addition, the retrieval module optionally supports multi-hop retrieval through iterative query rewriting [60, 78]. In this mode, the system determines whether the currently retrieved evidence is sufficient to answer the question. If not, the system rewrites the query using only phrases extracted from retrieved evidence while preserving key identifiers and structural references from the original question. Overall, the system consists of PDF ingestion and text preparation, chunking and indexing, evidence retrieval and ranking, context packing and LLM-based answer generation, and evaluation. Implementation details of each component and configuration settings are described in Chapter 5.

4.1 Data Collection

4.1.1 Standards PDF Corpus

The primary document corpus consists of automotive product safety standards provided as PDF files. All documents are English and contain extractable text, which

makes them suitable for text normalization, chunking, and indexing. PDFs that are not in English or contain only scanned images without extractable text are excluded. The corpus includes safety and compliance-relevant topics usually encountered in automotive engineering processes, including functional safety, cybersecurity, process assessment, and electrical disturbance testing. Representative examples include ISO-26262 (functional safety), ISO/SAE 21434, Automotive SPICE (process assessment), and ISO-7637 (electrical transient testing). These documents were selected because they are highly structured, use dense technical terminology, and include extensive cross-references, tables, and figures-properties that directly influence evidence retrieval in PDF-based RAG systems.

4.1.2 100 QA Set Construction

To evaluate the system on the target domain, a domain-specific set of 100 question-answer (100-QA) pairs is constructed from the selected automotive standards PDFs. Only publicly available standards PDFs are used; internal standards are excluded to prevent confidentiality and privacy risks. The QA set is constructed manually with support from domain experts in automotive product safety, based on their experience with compliance workflows and the structure of the standards documents. Table 4.1 shows three illustrative examples from the 100-QA dataset. To achieve domain-appropriate coverage, the evaluation set is constructed manually rather than relying on general-purpose publicly available benchmarks. The 100-QA dataset includes (i) natural-language questions, (ii) reference answers, and (iii) gold document labels used to compute precision, recall, and F1 at the document level.

Table 4.1.: Three illustrative examples from the 100-QA dataset

Questions	**Ground truth answer**	**Relevant PDF(s)**
According to ISO standards, what is the term for the organization that adds truck, bus, trailer, or semi-trailer bodies, cargo carriers, or equipment to a base vehicle?	body builder	`ISO_26262_1_2018.pdf`
Which supporting process defines an information management practice specifically for machine learning data and lists a base practice to assure quality of machine learning data?	Machine Learning Data Management	`Automotive_SPICE_PAM_v40.pdf`
What is the fundamental design guideline regarding insulation failures for electric materials in the power train?	Avoid insulation failures	`AIS_038.pdf`

4.1.3 FRAMES Benchmark

FRAMES [39] is a multi-hop reasoning benchmark built over Wikipedia articles and is designed to test whether retrieval systems can locate and integrate evidence distributed across multiple passages. To assess whether the proposed pipelines generalize beyond the automotive standards domain, evaluation is additionally performed on FRAMES. This setting differs from standards PDFs in both document structure and query style, making it suitable for cross-domain stress testing of retrieval strategies. In this thesis, FRAMES is used to analyze whether the relative behavior of baseline versus hybrid retrieval remains consistent when the corpus consists of general-knowledge text rather than technical standards documents. For retrieval evaluation, the document unit differs across datasets: in the standards setting, retrieved chunks map to source PDF files, whereas in FRAMES they map to Wikipedia articles. Table 4.2 presents sample FRAMES questions.

Table 4.2.: Sample FRAMES benchmark questions

ID	Question	Answer	Type	Evidence (short)
0	If my future wife has the same first name as the 15th first lady of the United States' mother and her surname is the same as the second assassinated president's mother's maiden name, what is my future wife's name?	Jane Ballou	Multiple constraints	`https://en.wikipedia.org/wiki/Harriet_Lane`, `https://en.wikipedia.org/wiki/James_A._Garfield`
1	How many years earlier would Punxsutawney Phil have to be canonically alive to have made a Groundhog Day prediction in the same state as the US capitol?	87	Numerical; Multiple constraints; Temporal	`https://en.wikipedia.org/wiki/Punxsutawney_Phil`, `https://en.wikipedia.org/wiki/United_States_Capitol`

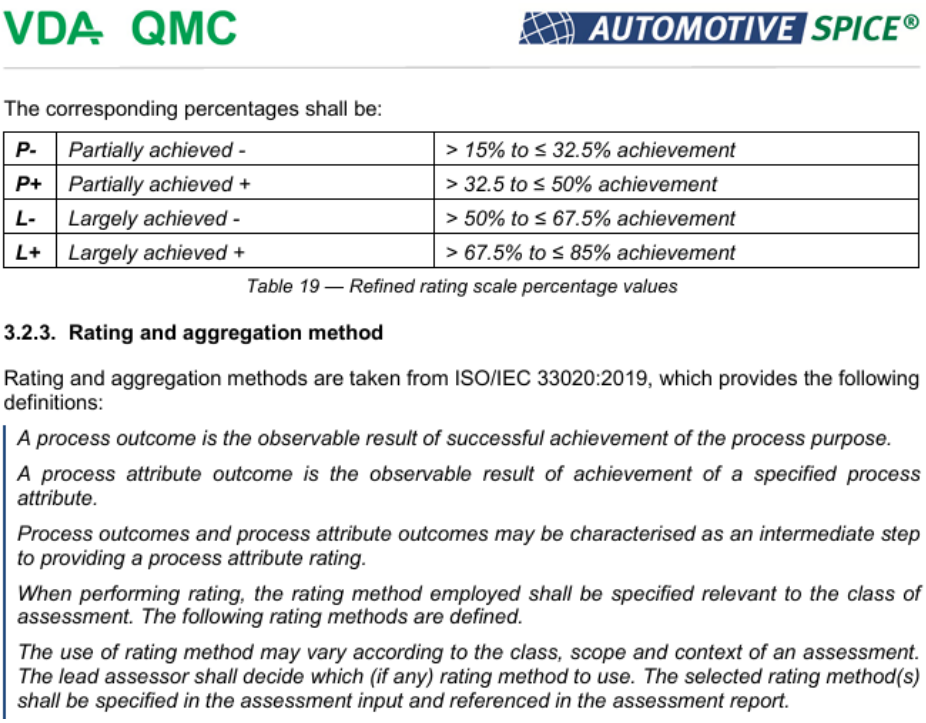
To keep retrieval evaluation aligned with benchmark provenance and computationally manageable, only the Wikipedia articles referenced by the selected FRAMES questions were indexed rather than the full Wikipedia snapshot. This setup provides a controlled comparison corpus, but it differs from open-web or full-Wikipedia retrieval settings and should therefore not be interpreted as equivalent to unrestricted retrieval.

4.2 Document Processing

4.2.1 The Challenge of Technical PDFs

The text extracted from product safety standards PDFs is frequently affected by layout and encoding-related artifacts that degrade chunk quality and reduce retrieval accuracy. In contrast to plain narrative documents, technical standards contain a mixture of structured and semi-structured content (e.g., tables, numbered clauses, figures, and cross-references) that is not preserved reliably by standard PDF text extraction. Technical standards documents present several challenges, as illustrated in Figure 4.1:

- **Repeated headers and footers:** Page headers, document titles, and page numbers are repeated across pages and introduce high-frequency noise that can dominate lexical retrieval.
- **Cross-references and section numbering:** Clauses frequently reference other sections (e.g., “5.3.2”, “Annex A”, “Table 7”), while the referenced information may appear far from the mention, increasing the need for robust evidence retrieval.
- **Multi-column layouts and hyphenation:** PDF extraction can change the order of columns and break words across lines, which make sentences and terms hard to read.
- **Tables and structured clauses:** Many documents includes tables, lists, or parameter grids that present requirements, such as limits and pass/fail criteria. Some of the meaning may be lost when it is converted into plain text, especially when the layout shows how the values are related.
- **Encoding inconsistencies and special symbols:** Units and symbols such as “μs”, “V” and other non-ASCII characters may be corrupted during extraction, which can reduce both lexical matching and embedding quality.



VDA QMC AUTOMOTIVE SPICE®

The corresponding percentages shall be:

P-	*Partially achieved -*	*> 15% to ≤ 32.5% achievement*
P+	*Partially achieved +*	*> 32.5 to ≤ 50% achievement*
L-	*Largely achieved -*	*> 50% to ≤ 67.5% achievement*
L+	*Largely achieved +*	*> 67.5% to ≤ 85% achievement*

Table 19 — Refined rating scale percentage values

3.2.3. Rating and aggregation method

Rating and aggregation methods are taken from ISO/IEC 33020:2019, which provides the following definitions:

> *A process outcome is the observable result of successful achievement of the process purpose.*
>
> *A process attribute outcome is the observable result of achievement of a specified process attribute.*
>
> *Process outcomes and process attribute outcomes may be characterised as an intermediate step to providing a process attribute rating.*
>
> *When performing rating, the rating method employed shall be specified relevant to the class of assessment. The following rating methods are defined.*
>
> *The use of rating method may vary according to the class, scope and context of an assessment. The lead assessor shall decide which (if any) rating method to use. The selected rating method(s) shall be specified in the assessment input and referenced in the assessment report.*

Figure 4.1.: Example of Complex Layout in Technical PDF

The main preprocessing goal is to produce stable text suitable for retrieval by preserving clause identifiers, numeric thresholds, and requirement statements while removing systematic extraction artifacts. Therefore, the pipeline applies deterministic normalization rules to remove common extraction artifacts (e.g., page markers and repeated headers) and to reduce repeated boilerplate and repair common encoding issues before indexing. More complex issues such as table reconstruction and cross-reference resolution are treated as open challenges and are not fully addressed in the current implementation.

4.2.2 Parsing and Text Extraction

The pipeline begins by ingesting each standards PDF and extracting its textual content into page-level plain-text segments suitable for downstream retrieval. Each segment is annotated with its source document (e.g., the originating filename) to support document-level retrieval evaluation. During extraction, the goal is to preserve the main reading text (including headings and list-like structures) while reducing systematic noise such as repeated page markers and table-of-contents artifacts. Therefore, complex layouts (e.g., multi-column formatting, tables, and embedded figures) can still introduce ordering and formatting noise. After extraction,

normalization is applied at the page level before chunking, and the cleaned text is then forwarded to the chunking stage.

4.2.3 Normalization Logic

Normalization is applied to the extracted text to reduce noise and stabilize downstream retrieval. The implemented normalization steps include:

- **Character and whitespace normalization:** PDF extraction can introduce irregular whitespace and artificial line breaks, which fragment technical terms and reduce lexical matchability during retrieval. Normalization combines the cleaned lines into a continuous text stream, fixes words that are split across line breaks, removes all broken spacing between words and combines repeated spaces into single space.
- **Pattern-based noise filtering:** This method removes all non-content lines using deterministic pattern rules targeting common PDF artifacts. This include explicit page labels (e.g., “Page X” or “Page X of Y”), numeric-only lines (often page numbers), and decorative separator lines consisting mainly of repeated symbols (e.g., “——”, “====”, “.....”).
- **Encoding correction:** Extraction may introduce corrupted Unicode characters and inconsistent encodings. An encoding-repair step is applied to reduce corrupted symbols and improve consistency of unit and notation tokens.

These rules are deterministic to ensure that observed differences in retrieval quality are attributable to retrieval strategy rather than preprocessing variation.

4.2.4 Chunking

After extraction, documents are split into overlapping text chunks to support efficient retrieval. Chunking decomposes long standards documents into smaller segments so that retrieval can return focused evidence passages rather than full pages. A fixed-length sliding window with overlap is used to preserve local context across chunk boundaries. It reduces the risk that a requirement statement and its conditions are separated into different chunks.

Chunking parameters are fixed throughout all experiments. The chunk size and overlap are chosen to reflect the structure of technical standards, where many requirements span multiple sentences and may include conditions, exceptions, or numeric thresholds. The overlap increases the likelihood that related sentences

co-occur in at least one chunk, which improves the chance that the retriever surfaces self-contained evidence for answer generation. The output of this step is a list of chunks per document, which serves as the input to indexing for both sparse (BM25) and dense retrieval. Chunking is necessary because full documents are often too long to index and retrieve effectively as single units, and because only a limited amount of retrieved text can later be passed to the generator within its context window. The same chunking principle also applies to Wikipedia articles used in the FRAMES evaluation pipeline, as full articles are too long and heterogeneous to be retrieved and passed to the generator as single units.

4.2.5 Indexing

The resulting chunks are indexed to enable efficient retrieval at query time. Indexing constructs searchable data structures over chunk texts, allowing the system to return the most relevant evidence passages for a given question. In the baseline system, a sparse lexical index is built over all chunks to support BM25-style term matching; the index stores each chunk's text together with metadata (e.g., the source document identifier). In the hybrid system, an additional dense vector index is built over the same chunks by embedding each chunk and storing the vectors, together with the same metadata, in a nearest-neighbor index for semantic retrieval. Both indexing schemes retain the originating source document identifier as metadata, which enables mapping retrieved chunks back to their source PDFs (or Wikipedia articles) for document-level evaluation.

4.3 Retrieval Strategy

The retrieval component selects relevant chunks from the indexed corpus and provides evidence used during answer generation. Two retrieval configurations are evaluated over the same chunk set: a baseline and a hybrid retriever.

Formally, let $D = \{d_1, \ldots, d_n\}$ denote the document corpus and let $C = \{c_1, \ldots, c_m\}$ denote the chunk set produced after parsing and chunking. Given a query q, sparse retrieval returns a ranked list $L_s(q)$, dense retrieval returns a ranked list $L_d(q)$, and hybrid retrieval produces a fused ranking $L_f(q)$. From the final ranking, the system selects a context $C(q) \subseteq C$ for answer generation, and the language model produces an answer a conditioned on q and $C(q)$.

BM25 is used as the baseline. Dense retrieval is not evaluated as a standalone primary configuration in this thesis; instead, it is treated as a complementary signal within the hybrid pipeline. This design reflects common RAG practice, where sparse retrieval provides high-precision lexical matches while dense retrieval can improve recall for paraphrases and semantically related formulations.

4.3.1 Sparse Retrieval (BM25)

In the sparse setting, illustrated in Figure 4.2, BM25 ranks chunks by lexical matching: chunks receive higher scores when query terms occur frequently in the chunk and are rare in the overall collection. This retrieval mode prioritizes exact word overlap between the query and chunk text, which is especially useful for technical standards queries containing identifiers, clause references, and domain-specific terminology. In the baseline configuration, the retriever returns the highest-ranked BM25 chunks, which are forwarded to context construction.

In the hybrid configuration, BM25 retrieves a larger candidate set that is later combined with dense candidates during fusion. BM25 is used as the baseline because it is a widely adopted sparse retrieval method. It provides a transparent lexical reference point for comparison. It also provides a strong reference point for identifier-heavy standards queries. In such queries, exact matching of clause identifiers, technical terms, and numeric tokens is important.

4.3.2 Dense Retrieval (Embedding Similarity)

Dense retrieval represents each chunk as a dense vector embedding and retrieves chunks by similarity in the embedding space. Chunk embeddings are computed once for the full corpus and reused across queries, while only the query embedding is computed at runtime. The dense index uses L2-normalized embeddings and inner-product similarity, which is equivalent to cosine similarity [33] for normalized vectors. This enables retrieval based on semantic similarity beyond exact word overlap. In the hybrid configuration, dense retrieval returns a ranked candidate set of semantically similar chunks.

L2-normalization scales vectors to unit length, which stabilizes similarity comparisons across queries and chunks. With normalized vectors, inner product similarity corresponds to cosine similarity, which is a common choice for embedding-based retrieval [33]. The embedding model depends on the deployment mode; the exact

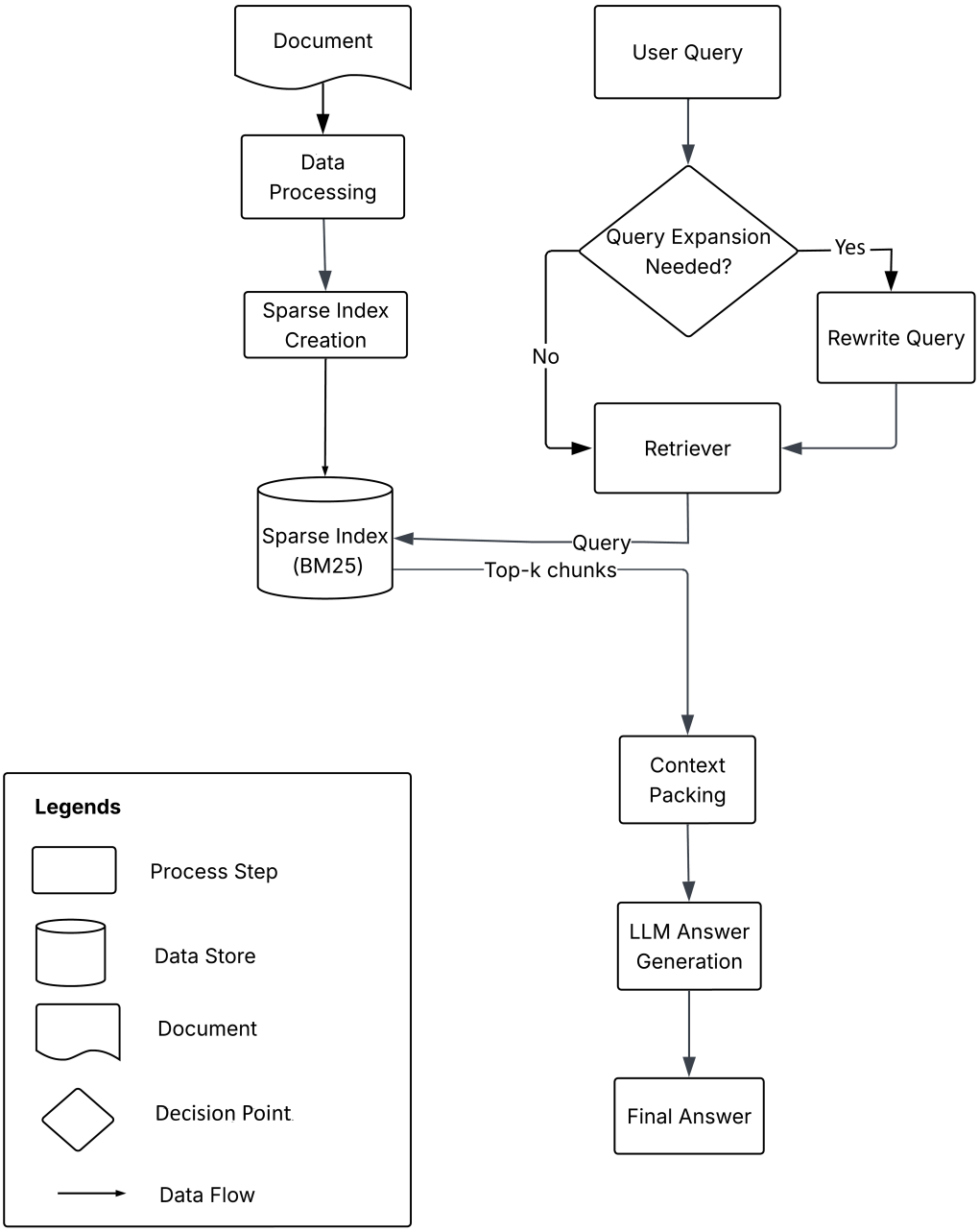


Figure 4.2.: Baseline BM25 retrieval pipeline

model configuration is described in Chapter 5. A dense-only configuration was not selected as a main comparison because the target standards queries often include identifiers, clause references, and exact terminology for which lexical matching remains especially important.

4.3.3 Hybrid Retrieval

Hybrid retrieval is implemented as a multi-stage procedure: independent candidate retrieval, fusion, and optional reranking, multi-hop retrieval, and query expansion. Figure 4.3 summarizes the hybrid retrieval, fusion, and reranking workflow.

Step 1: Independent retrieval of sparse and dense candidates Given a query q, BM25 produces a ranked sparse candidate list L_s, and dense retrieval produces a ranked dense candidate list L_d.

Step 2: Rank fusion using Reciprocal Rank Fusion (RRF). Hybrid retrieval produces two ranked lists, but the underlying scores are on different scales (BM25 scores vs. embedding similarity). Therefore, RRF is used to combine rankings instead of raw scores [13]. RRF does not require score calibration and is robust when a relevant chunk is highly ranked by only one retriever. For each chunk c, the fused score is:

$$\mathrm{RRF}(c) \;=\; \sum_{L \in \{L_s, L_d\}} \frac{1}{k_{\mathrm{rrf}} + \mathrm{rank}_L(c)} \tag{4.1}$$

[13] where $\mathrm{rank}_L(c)$ is the 1-indexed rank of chunk c in list L, L_s and L_d denote the sparse and dense ranked lists, and k_{rrf} is a smoothing constant that reduces the influence of lower-ranked items. Candidates are sorted by $\mathrm{RRF}(c)$ to produce a unified ranking L_f. When reranking is enabled, the fused ranking is truncated to a fixed candidate set size before cross-encoder scoring; otherwise, final evidence is selected directly from the fused ranking. RRF was selected instead of weighted score fusion because it does not require score normalization or validation-set tuning. This is making it better suited to a controlled comparison across datasets.

Step 3: Re-ranking of the candidate set. To isolate the impact of reranking, the hybrid pipeline is evaluated in two modes: (i) reranking disabled, where final evidence is selected directly from L_f, and (ii) reranking enabled, where a cross-encoder reranker re-scores query-chunk pairs for a fixed fused candidate set. Reranking is

intended to improve precision at the top of the list by applying a stronger relevance model that jointly considers the full query and the full chunk text, which can correct ranking errors made by BM25 and embedding similarity. In a RAG pipeline, reranking is typically applied after the initial retrieval stage and before final context construction, so that only the most relevant candidates are passed to the generator.

4.3.4 Query Normalization and Expansion

During retrieval, the user query is normalized and expanded using a domain-specific abbreviation and unit mapping maintained in a reference dictionary. The mapping is created manually with support from automotive product safety experts. It is based on terminology found in the selected standards corpus (e.g., abbreviation tables and definitions). Figure 4.4 illustrates the type of abbreviation evidence used during curation. For BM25, the query is first normalized into a parser-compatible form and then expanded using bidirectional rules that map abbreviations to their long forms and vice versa. The purpose of this step is to reduce vocabulary mismatch between the user query and standards terminology while limiting expansion noise. The expanded BM25 query is constructed as a structured boolean form, combining the original query with controlled clauses of the form `(term OR expansion)`; expansions are deduplicated and capped to limit noise, and unit/symbol-like expansions are only applied when the query contains numeric and unit-hint context.

For dense retrieval, the query is rewritten into a compact text form: abbreviations are replaced with long forms (longer keys first to avoid partial overlaps), and a small number of parenthetical abbreviations may be appended. Lightweight question scaffolding is removed to keep the embedding input short and to avoid embedding noise. BM25 works better when the query includes both the short form and full form used in the documents, while dense retrieval can already connect similar meanings, so keeping the query shorter helps avoid confusion. The mapping is iteratively refined using candidate abbreviations from the corpus and expert feedback. Validated expansions are added, and ambiguous entries that could introduce query drift are removed.

4.3.5 Multi-hop Retrieval

Multi-hop retrieval is evaluated in two settings, as illustrated in Figure 4.5 with multi-hop enabled and with multi-hop disabled. It extends single-step retrieval by allowing additional retrieval rounds when the initial evidence is insufficient.

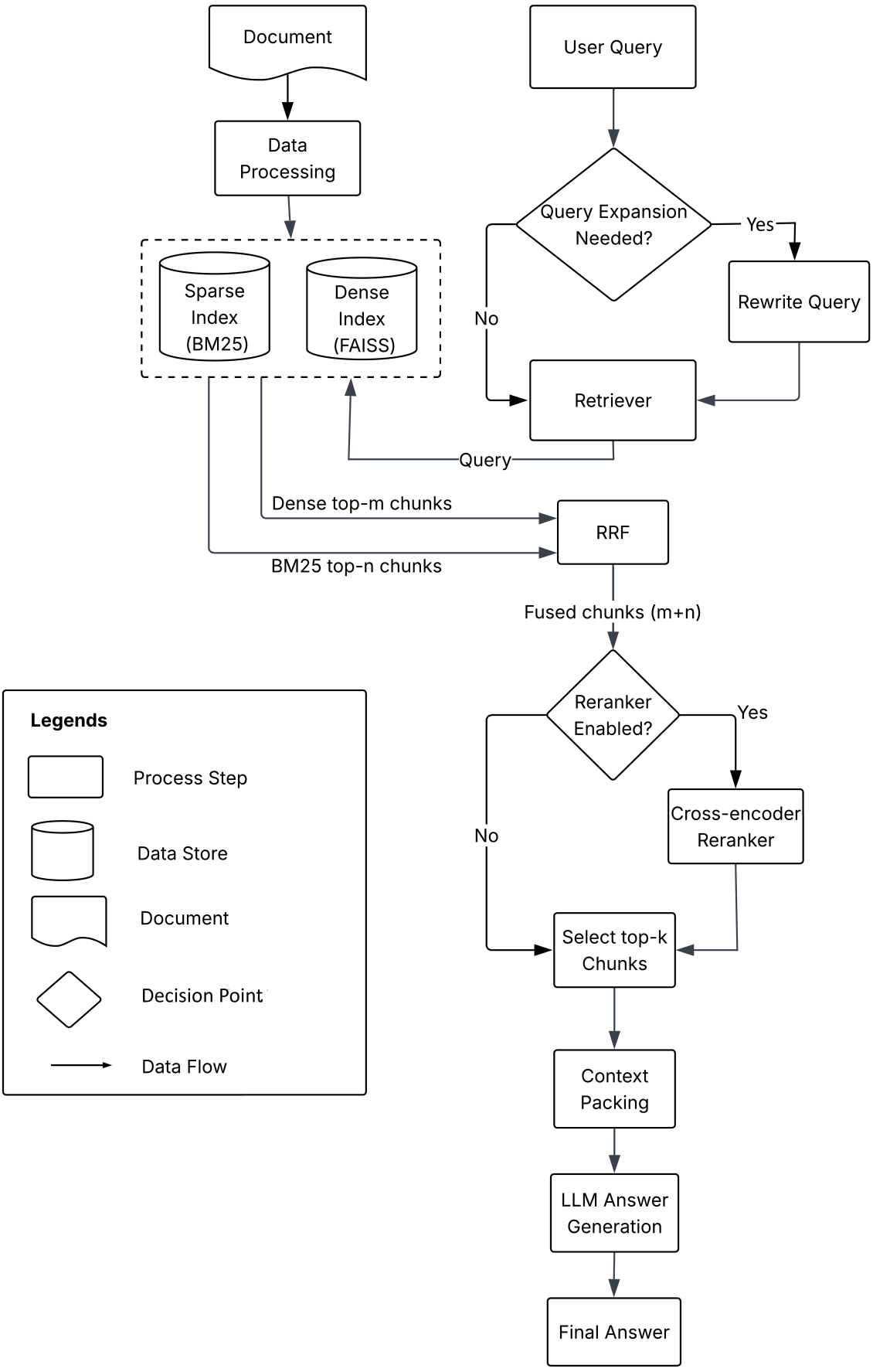


Figure 4.3.: Hybrid pipeline

VDA QMC AUTOMOTIVE SPICE®

1.3. Abbreviations

BP	**B**ase **P**ractice
CAN	**C**ontroller **A**rea **N**etwork
CASE	**C**omputer-**A**ided **S**oftware **E**ngineering,
CCB	**C**hange **C**ontrol **B**oard
CPU	**C**entral **P**rocessing **U**nit
ECU	**E**lectronic **C**ontrol **U**nit
EEPROM	**E**lectrically **E**rasable **P**rogrammable **R**ead **O**nly **M**emory
EOL	**E**nd-**o**f-**L**ine
FMEA	**F**ailure **M**ode and **E**ffect **A**nalysis
FTA	**F**ault **T**ree **A**nalysis
GP	**G**eneric **P**ractice
GR	**G**eneric **R**esource
IEC	**I**nternational **E**lectrotechnical **C**ommission
IEEE	**I**nstitute of **E**lectrical and **E**lectronics **E**ngineers
I/O	**I**nput / **O**utput
ISO	**I**nternational **O**rganization for **S**tandardization
LIN	**L**ocal **I**nterconnect **N**etwork

Figure 4.4.: Example abbreviation table from a standards document

When enabled, the process starts from the original query and performs iterative expansions for multiple hops, up to a predefined limit. At each hop, the selected retriever (BM25-only or hybrid) returns candidate chunks. An LLM-based judge then assesses whether the currently retrieved evidence is sufficient to answer the question. If the evidence is sufficient, the procedure terminates early. This early stopping condition keeps the process iterative while avoiding unnecessary hops when the initial evidence is already sufficient.

If the evidence is insufficient, the query is rewritten using a small set of high-signal keywords and phrases extracted directly from the retrieved evidence passages. To avoid drifting away from the user's intent, identifiers from the original question are kept unchanged, such as standard IDs and clause, table, or annex references. To further reduce query drift, the process stops early when retrieval returns no documents, when successive hops return almost the same top-ranked passages, or when the rewritten query is not meaningfully different from an earlier one. To decide whether another hop is needed, the system performs a separate answerability check. It sends the original question together with up to six of the highest-ranked retrieved passages to the model. These passages are combined into a short evidence block before evaluation. The model is instructed to use only the provided evidence, make a strict binary decision, and return `NO` when the evidence is insufficient to answer the

question directly. If important information is still missing, it returns `NO`. The output is returned in reference dictionary. It contains the decision, what is missing, and a short reason. This output is used only to control the multi-hop process. A `YES` result stops retrieval, while a `NO` result leads to query rewriting and another hop. The exact prompt template used for this answerability check is provided in Appendix A.3.4.

More complex reasoning-agent strategies were not used here because they would add additional planning variables and make it harder to isolate the effect of iterative retrieval itself.

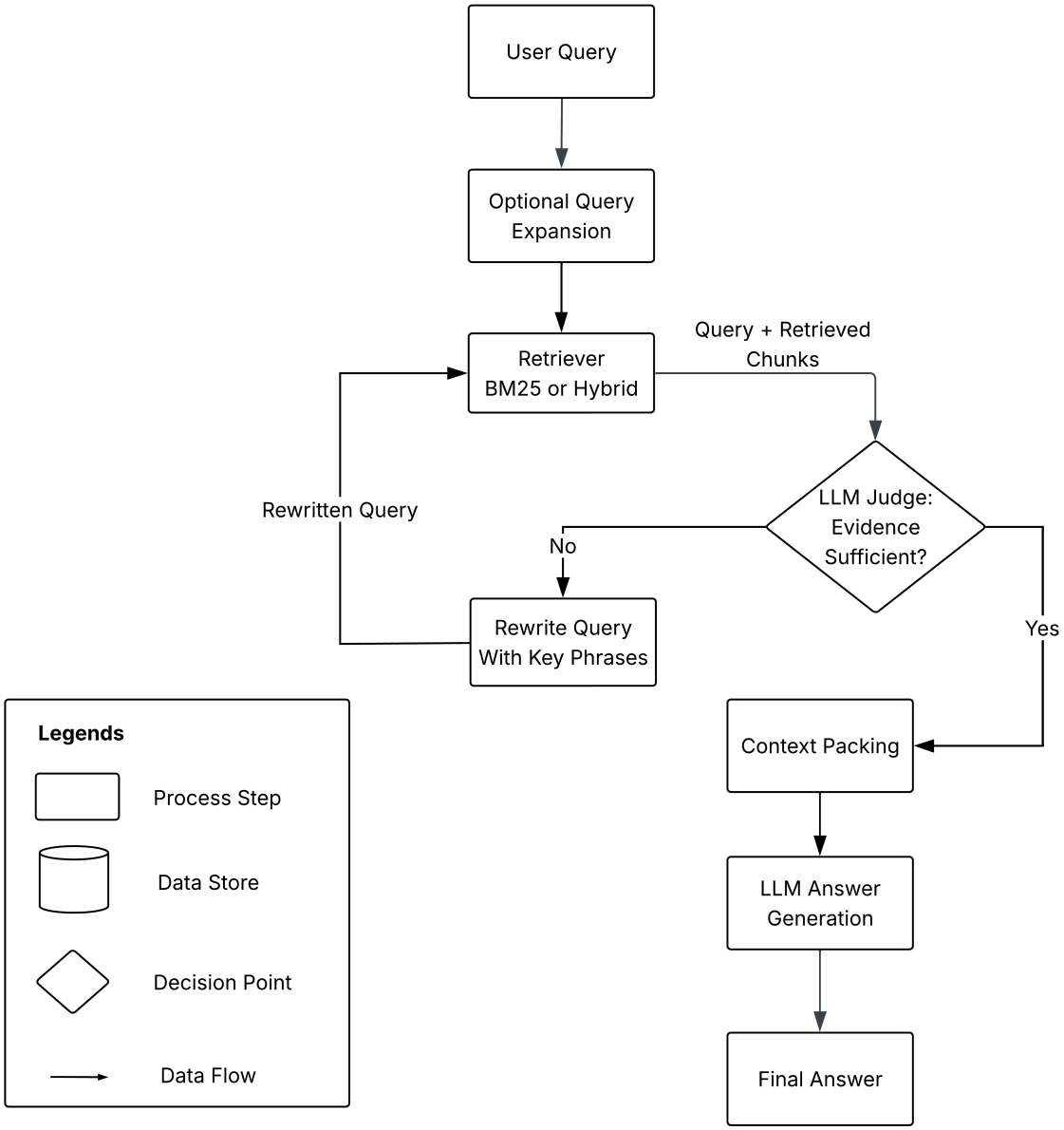


Figure 4.5.: Multi-hop Retrieval Workflow

4.3.6 Context Packing and Answer Generation

In all configurations, the final ranked evidence list is converted into a single LLM input context by concatenating the selected chunks in rank order, separated by double newlines. This produces a fixed-format context string that is passed to the answer generation module. To ensure a controlled comparison between retrieval strategies, a fixed prompt template is used within each evaluation setting. The prompt enforces evidence grounding: the model must answer using only the provided context, keep responses brief, and return `I do not know` when the evidence is insufficient. The prompt templates used for answer generation are provided in Appendix A.3.

4.4 Evaluation

The retrieval framework is evaluated on both datasets: 100 QA and FRAMES. The compared system variants are the BM25 baseline, hybrid retrieval without reranking, and hybrid retrieval with reranking, while query expansion and multi-hop retrieval are analyzed as optional retrieval enhancements. Two metric groups are used: retrieval-level metrics and answer-level metrics. The retrieval-level metrics assess whether the system finds the right source documents. While answer-level metrics examine whether the generated response is correct and grounded in the retrieved evidence.

4.4.1 Retrieval Metrics

Retrieval performance is measured at the document level using precision, recall, and F1 scores. For each question, the system evaluates the retrieved chunks that are forwarded to context construction. Each retrieved chunk includes metadata indicating its source document (PDF name for standards, Wikipedia article for FRAMES). The retrieved document set D_{ret} is formed by mapping chunks to their source documents and removing duplicates. It is then compared to the gold-standard document set D_{gold}, which contains the relevant documents for the question.

Here, D_{ret} is the set of unique source documents represented by the retrieved chunks, and D_{gold} is the set of gold relevant documents for the query.

$$P = \frac{|D_{\text{ret}} \cap D_{\text{gold}}|}{|D_{\text{ret}}|}, \quad R = \frac{|D_{\text{ret}} \cap D_{\text{gold}}|}{|D_{\text{gold}}|}, \quad F_1 = \frac{2PR}{P+R} \tag{4.2}$$

Here,

- **Precision:** P denotes as the proportion of retrieved documents that are relevant to the query.
- **Recall:** R denotes as the proportion of relevant documents that are successfully retrieved.
- **F1:** F_1 dentoes the harmonic mean of precision and recall, providing a single summary of retrieval effectiveness.

The retrieval metrics are aggregated across all questions to provide an overall assessment of retrieval performance for each configuration.

4.4.2 Generation Metrics

End-to-end answer quality is evaluated using RAGAS under the same context budget used for retrieval comparison.

The following metrics are used:

- **Context precision** measures the relevance of the retrieved context.
- **Faithfulness** measures whether the generated answer is supported by the provided context.
- **Answer correctness** measures agreement with the reference answer.

These metrics complement retrieval scores by assessing whether the retrieved evidence led to grounded and correct answer generation.

5 Implementation

This chapter focuses on the implementation of the system components, the data flow between modules, and the practical realization of the two retrieval configurations. The system is implemented in two runnable configurations: a BM25 baseline and a hybrid configuration that extends the baseline with dense retrieval. The methodological choices are described in Chapter 4. This chapter focuses on how these components are implemented and how data flows between modules under different runtime configurations. The full implementation of the system is available in the github repository referenced in Appendix A.2.

5.1 System Overview

5.1.1 Technologies and Packages

All components are implemented in Python programming language. Sparse retrieval is implemented with Whoosh using BM25 scoring. The hybrid configuration additionally uses FAISS [15] for dense indexing and optional cross-encoder reranking. Table 5.1 lists the main libraries used in the implementation.

Table 5.1.: Main Python packages used in the implementation.

Package	**Use / Description**
`langchain_community`	PDF loading (`PyPDFLoader`) and document handling.
`ftfy`	Fixes encoding issues in extracted PDF text.
`whoosh`	Sparse BM25 indexing and retrieval.
`faiss`	Dense vector indexing for the hybrid setup.
`sentence_transformers`	Optional cross-encoder reranker.
`tiktoken`	Token counting for context limits.
`openai`	Chat completions (Azure/OpenAI endpoints).
`datasets`	Loads the FRAMES benchmark dataset.
`ragas`	RAG evaluation metrics.
`pandas`	Data handling and results storage.
`numpy`	Numeric operations for retrieval and scoring.

These tooling choices were selected because they provide lightweight, well-documented support for the core requirements of the thesis. PDF ingestion, lexical and dense retrieval, configurable generation backends, and reproducible evaluation without introducing unnecessary system complexity. Although several frameworks exist for building RAG pipelines, this implementation uses LangChain as a lightweight integration layer for document loading and handling. In particular, LangChain provides a convenient interface for PDF ingestion through PyPDFLoader. PyPDFLoader is a library used to load PDF files and extract their text. It supports document objects with attached metadata, which simplifies downstream chunking and traceability. This reduced implementation effort and allowed the retrieval and generation pipeline to be built in a modular way, while the main retrieval components themselves were implemented explicitly with Whoosh, FAISS, and custom runtime logic.

5.1.2 Runtime Configuration

Experiments are controlled through environment variables and a shared configuration dictionary. This configuration specifies the document mode, chunking settings, index locations, retrieval cutoffs, and model endpoints. It also controls optional modules such as abbreviation-based query expansion, multi-hop retrieval, and reranking. This allows the same codebase to be executed under different experimental settings without changing the core pipeline logic.

Table 5.2.: Main runtime parameters used in the experiments

Parameter	Value	Description
PDF chunk size	800	Characters per chunk for standards PDFs
PDF chunk overlap	150	Overlap between adjacent PDF chunks
FRAMES chunk size	1500	Characters per chunk for Wikipedia articles
FRAMES chunk overlap	300	Overlap between adjacent FRAMES chunks
BM25 top-k	30	Number of sparse candidates retrieved per query
Dense top-k	30	Number of dense candidates retrieved per query
Fused candidate cutoff	40	Number of fused candidates retained before final selection or reranking
Final context top-k	10	Number of chunks passed to answer generation
Reranker batch size	10	Batch size used during cross-encoder reranking
BM25 max query expansions	10	Maximum number of controlled glossary-based expansions
Max hops	3	Maximum number of multi-hop retrieval iterations

Deployment mode and model selection are likewise configurable through environment variables. As a result, the system can run with either cloud-based or local model. Answer generation was performed with deterministic decoding using a temperature of 0 to ensure consistent and reproducible outputs. This setting reduces randomness in token selection, so the model produces the most likely answer given the same input.

5.2 PDF Ingestion and Text Normalization

5.2.1 PDF Loading

PDFs are loaded and parsed into page-level text objects. This is done using LangChain's PyPDFLoader. This provides a simple interface for extracting text and page metadata from PDF files. The implementation iterates over all PDF files in the configured directory and processes each page independently. For each page, the extracted text and the page number are passed to the normalization and chunking stages. At chunk creation time, the implementation preserves provenance metadata including the PDF filename as `source_file` and the page number as `page`. This enables document-level retrieval evaluation and traceability by mapping retrieved chunks back to their original PDF and page.

5.2.2 Text Cleaning and Normalization

The normalization rules are defined in Chapter 4. In the implementation, normalization is applied at the page level before chunking.

Algorithm 5.1 is executed for each extracted PDF page before the text is divided into chunks. It first repairs formatting artifacts introduced during PDF extraction. This includes inconsistent line breaks and words that were split across lines. It then removes lines that do not contribute useful content for retrieval, including empty lines, page headers or footers, isolated page numbers, and separator-only lines. The remaining lines are merged into a continuous text segment, whitespace is normalized, and Unicode cleanup is applied using ftfy. The ftfy is a library for cleaning broken Unicode text and fixing character-encoding issues. In practice, this preprocessing reduces indexing noise. It also improves the quality of the chunks passed to both the BM25 and hybrid retrieval pipelines.

Algorithm 5.1: Text normalization applied during PDF ingestion

Function *NormalizeText(text)*:
    Replace carriage return characters with newline characters;
    Repair words split across line breaks;
    Initialize $cleaned_lines$ as an empty list;
    foreach *line in text split at newline boundaries* **do**
        $s \leftarrow \mathrm{strip}(line)$;
        if s *is empty* **then**
            continue;
        if s *corresponds to pagination metadata* **then**
            continue;
        if s *is a standalone numeric token* **then**
            continue;
        if s *corresponds to a separator artifact* **then**
            continue;
        Append s to $cleaned_lines$;
    $text \leftarrow$ concatenate $cleaned_lines$ using single spaces;
    Remove intra-word spacing artifacts;
    Collapse consecutive whitespace characters;
    Apply Unicode normalization;
    return $text$;

5.3 Chunking and Indexing

5.3.1 Chunking

Chunking is implemented as fixed-size character windows with overlap, using configurable values for chunk length and overlap size. The chunk size and overlap are controlled through configuration settings, with separate values available for the PDF and FRAMES datasets. The resulting chunks are stored as lightweight document objects together with metadata required for indexing and evaluation. Each chunk preserves provenance information, including `source_file` and `page`. In addition, a stable `chunk_id` is assigned and the character offsets `char_start` and `char_end` are stored to support debugging, traceability, and document-level evaluation.

5.3.2 Indexing

Sparse indexing (BM25 / Whoosh) For sparse retrieval, the system builds a Whoosh BM25 index over the chunk texts. Each chunk is stored with a unique `doc_id`, the originating `source_file`, a normalized label `source_norm`, and the chunk `content`. This index supports lexical matching and enables ranked retrieval over the chunk collection for a query.

```
schema = Schema(
    doc_id=ID(stored=True, unique=True),
    source_file=ID(stored=True),
    source_norm=ID(stored=True),
    content=TEXT(stored=True, analyzer=StemmingAnalyzer()),
)
```

Listing 5.1: Whoosh schema for BM25 sparse index.

This schema5.1 defines the minimal fields needed for sparse retrieval and traceability. To improve lexical matching, the `content` field is indexed with a stemming analyzer. The `doc_id`, `source_file`, and `source_norm` are stored so retrieved results can be traced back to their original documents.

Dense indexing (Embeddings / FAISS) In the hybrid configuration, each chunk is also encoded into a dense vector representation as shown in 5.2 and stored in a FAISS inner-product index. Embeddings are normalized during generation, so inner-product similarity is equivalent to cosine similarity. Alongside the vector index,

the implementation persists metadata for each chunk. This metadata includes its document identifier, source file, normalized source label, page number, character offsets, and text content, so that FAISS hits can be mapped back to their original chunks during retrieval.

Algorithm 5.2: Dense index construction

Function *BuildDenseIndex(chunks)*:
  $texts \leftarrow$ extract text from each chunk;
  $vectors \leftarrow \text{Encode}(texts)$;
  $vectors \leftarrow \text{L2Normalize}(vectors)$;
  $vectors \leftarrow \text{Float32}(vectors)$;
  $index \leftarrow \text{FaissIndexIP}()$;
  $index.\text{add}(vectors)$;
  return $index$;

This procedure converts each chunk into a normalized dense vector and inserts the resulting vectors into a flat inner-product FAISS index. Because embeddings are normalized before insertion. This means inner-product search effectively becomes cosine similarity, which is used as the dense part of the hybrid pipeline.

5.4 Retrieval Module Implementation

This section describes how sparse (BM25) and hybrid retrieval are implemented on top of the chunked corpus.

5.4.1 Sparse Retrieval

Sparse retrieval algorithm 5.3 is implemented with a Whoosh BM25 index. At query time, the retriever optionally applies BM25-oriented query expansion and then parses the resulting query string with a Whoosh query parser. The search returns the top-k hits, each mapped into a lightweight document object that contains the stored chunk text and provenance metadata. For document-level evaluation or constrained settings such as FRAMES, the retriever can optionally restrict results to a predefined set of allowed sources. This is implemented by filtering over the stored `source_norm` field. The final BM25 query string is also recorded in a debug dictionary for traceability.

Algorithm 5.3: BM25 retrieval with optional query expansion and source filtering

Function *BM25Retrieve(query, allowed_sources, top_k)*:
    `// Optionally expand abbreviations or shorthand forms in the query`
    if *query expansion is enabled* **then**
        $query \leftarrow \text{ExpandQuery}(query, abbrev_map, max_expansions)$;
    `// Parse the query into the representation required by the BM25 search engine`
    $parsed_query \leftarrow \text{Parse}(query)$;
    `// Initialize the source filter as empty`
    $source_filter \leftarrow \varnothing$;
    `// Restrict retrieval to the selected sources, if provided`
    if $allowed_sources$ *is not empty* **then**
        $source_filter \leftarrow$ disjunction of $\text{Term}(\textit{source_norm}, s)$ for all $s \in allowed_sources$;
    `// Execute BM25 retrieval and return the highest-ranked results`
    $results \leftarrow \text{BM25Search}(parsed_query, limit = top_k, filter = source_filter)$;
    return $results$;

If direct parsing fails, a fallback parser cleans problematic punctuation and retries the query.

5.4.2 Dense Retrieval

Dense retrieval algorithm 5.4 is implemented using a FAISS inner-product index over chunk embeddings. At query time, the user query is optionally rewritten using the dense query expansion function. The rewritten query is embedded using the same embedding model configuration that was used during indexing. The FAISS index is searched for the top-k nearest vectors. Each FAISS hit is identified by its vector position, which is used to recover the corresponding chunk metadata. Optional source filtering is applied using the stored `source_norm` field.

The final rewritten dense query is also stored in a debug dictionary so that it can be traced.

Algorithm 5.4: Dense retrieval with optional query rewriting and FAISS search

Function *DenseRetrieve(query, allowed_sources, top_k)*:
    `// Apply optional query rewriting for dense retrieval`
    if *query expansion is enabled* **then**
        $query \leftarrow \text{ExpandQueryDense}(query, abbrev_map)$;
    `// Encode the query into the dense vector space`
    $query_vector \leftarrow \text{Encode}(query, normalize = \text{True})$;
    `// Perform nearest-neighbor search over the FAISS index`
    $(scores, indices) \leftarrow \text{FAISSSearch}(query_vector, limit = top_k)$;
    Initialize $results$ as an empty list;
    foreach *retrieved position* idx *with rank* $rank$ *in* $indices$ **do**
        if idx *is invalid* **then**
            continue;
        $metadata \leftarrow \text{MetadataLookup}(index_position = idx)$;
        if $metadata$ *is not found* **then**
            continue;
        if $allowed_sources$ *is not empty* ***and*** $metadata.source_norm \notin allowed_sources$ **then**
            continue;
        Append the retrieved document with score $scores[rank]$ to $results$;
    return $results$;

5.5 Hybrid Retrieval

The hybrid retriever retrieves two ranked candidate lists: BM25 candidates and dense candidates. These lists are then fused, after which the fused ranking is either passed directly to final selection or truncated to a fixed candidate set before reranking.

5.5.1 Reciprocal Rank Fusion (RRF)

Rank fusion is implemented using a fixed RRF constant ($k = 60$). The fused score is stored in each returned document's metadata as `rrf_score`. Fusion is performed on document identifiers, so a chunk receives a higher fused score when it is ranked highly by both retrieval branches. The RRF constant was set to k=60, as suggested in the original RRF formulation, to combine the BM25 and dense rankings smoothly [13].

5.5.2 Cross-Encoder Reranking

After fusion, reranking is optionally applied using a Sentence-Transformers cross-encoder. The reranker is enabled or disabled via configuration setting. When enabled, the reranker scores each query, chunk pair. It stores the score as `rerank_score` in the returned metadata. Candidates are then sorted by this score. Finally, the top `hybrid_top_k` results are returned. The default cross-encoder model is `cross-encoder/ms-marco-MiniLM-L6-v2`. During initialization, the implementation resolves the reranker model from the configured runtime settings.

5.5.3 Domain-Specific Query Expansion

Query expansion is implemented as an optional preprocessing step inside the retrievers. Separate runtime flags control whether expansion is enabled for the BM25 and dense retrieval branches. The abbreviation glossary is loaded from a JSON file specified in the configuration. During loading, glossary keys are normalized to lowercase so that matching is case-insensitive. When expansion is enabled, the retriever replaces the original user query with an expanded or rewritten query and stores a debug payload describing the applied transformations.

BM25-Friendly Expansion

For sparse retrieval, the implementation constructs a structured BM25 query string from the input query and the matched glossary entries. The query is checked against abbreviations and full-form pairs, and the final query is built for BM25 retrieval. Unit and symbol-related expansions are only applied when the query contains compatible numeric or unit context. The retriever stores both the final BM25 query string and the applied expansions in its debug output.

Dense-Friendly Expansion

For dense retrieval, the implementation rewrites the input query into a normalized dense-retrieval query string. Abbreviations are replaced with their long forms, with longer matches applied first to avoid partial replacements. A limited number of abbreviations may also be appended in parentheses when present in the glossary. After rewriting, lightweight question scaffolding is removed and whitespace is normalized. The retriever stores the final rewritten dense query and the applied replacements in its debug output.

5.5.4 Multi-hop Retrieval

Multi-hop retrieval is implemented as an optional wrapper around the retriever's `invoke` method. When multi-hop retrieval is enabled, the wrapper repeatedly calls the underlying retriever with an updated query for a bounded number of hops. At each hop, the current query is executed and the retrieved chunks are collected as evidence. An LLM is used as judge to check whether the original question is answerable from the retrieved evidence alone.

If the evidence is judged insufficient, the implementation extracts high-signal keywords and short phrases from the retrieved chunks. It then rewrites the query using only these evidence-grounded phrases together with preserved anchors from the original question, such as identifiers or clause markers. This constrains rewriting to observed evidence and reduces query drift across hops.

The iteration stops early when one of the implemented stopping conditions is met. These include the situations where no documents are found, retrieval does not get better in a meaningful way, the evidence is enough to answer the question, no keywords are extracted, the query does not change meaningfully, or the maximum

number of hops is reached. After that, the retrieved chunks from all hops are deduplicated and returned as the final evidence set. The wrapper also returns a debug object containing per-hop information. This information contains the executed query, stop reason, extracted keywords, rewritten query, and the debug output from the underlying retriever.

5.6 Context Packing Implementation

Context packing is implemented as a simple concatenation of the retrieved chunk texts in rank order. Before concatenation, empty or whitespace-only chunk texts are removed. The resulting context string is constructed using double newline separators and is passed directly to the generation function. In the current implementation, context length is controlled upstream through chunking and retrieval configuration rather than by a token-budget check inside the packing routine.

5.7 Answer Generation

Answer generation is implemented through a unified client interface that supports both Azure and local OpenAI-compatible endpoints. For each question, the system passes the packed retrieval context and the user query to the generator with an instruction to answer only from the provided evidence and to avoid unsupported claims. The generated answer is returned together with the retrieved contexts for later evaluation and error analysis.

5.8 Reproducibility and Practical Considerations

The implementation includes several practical mechanisms to support stable experimentation. Configuration flags control whether sparse and dense indexes are rebuilt or reused across runs. Debug information from retrieval and query expansion is stored alongside the results for later analysis. The runtime configuration supports both OpenAI models and Open source model endpoints, enabling the same pipeline to be executed under different deployment settings. The same pipeline supports both proprietary cloud-hosted models and open-weight local models through the same generation interface.

5.9 Summary

This chapter described the implementation of the BM25 baseline and hybrid retrieval pipelines. It covered PDF ingestion, normalization, chunking, sparse and dense indexing, rank fusion, optional reranking, query expansion, multi-hop retrieval, context packing, and answer generation. The same execution and evaluation framework is used for both the standards QA dataset and the FRAMES dataset, supporting consistent comparison of retrieval configurations.

6 Evaluation

This chapter evaluates the BM25 baseline and the hybrid pipeline. The goal is to study how the retrieval strategy affects retrieval quality and answer quality. Results are reported for the standards-based 100-QA PDF dataset and for 300 questions from the FRAMES benchmark.

6.1 Setup

This section summarizes the evaluation setup used in this chapter. Implementation details are described in Chapter 5.

6.1.1 Comparative Analysis of Retrieval Strategies

This chapter compares two retrieval pipelines introduced in section 4.3:

- BM25 baseline
- Hybrid pipeline

For the standards-based 100-QA PDF dataset, the following configurations are evaluated:

- BM25 baseline
- Hybrid baseline
- BM25 with query expansion
- Hybrid with query expansion
- BM25 with multi-hop retrieval
- Hybrid with multi-hop retrieval
- Hybrid with reranking

For FRAMES, the following configurations are evaluated:

- BM25 baseline
- Hybrid baseline
- BM25 with multi-hop retrieval

- Hybrid with multi-hop retrieval
- Hybrid with reranking

In FRAMES, retrieval is done only over the Wikipedia articles provided by the benchmark as reference links, not the entire Wikipedia. Retrieval is evaluated by comparing retrieved article names with the ground-truth article names. This controlled setup ensures that retrieval can be fairly compared to the benchmark.

Query expansion is not evaluated on FRAMES because the expansion dictionary was built from domain-specific abbreviations and terminology in the standards corpus rather than from Wikipedia-based FRAMES documents. Reranking is evaluated only in the hybrid pipeline because it reorders the fused BM25 and dense candidate set after hybrid retrieval.

6.1.2 Datasets

Two datasets are used in the evaluation and the full dataset links are provided in Appendix A.1.

- **100-QA dataset:** This dataset contains 100 question-answer pairs created based on automotive standards PDFs. It is the main domain-specific evaluation setting in this thesis. The dataset construction is described in Section 4.1.2.
- **FRAMES:** FRAMES is used as a cross-domain benchmark. In this thesis, it is used to examine whether the observed retrieval behavior also appears outside the standards domain. The use of FRAMES in this thesis is described in Section 4.1.3.

The FRAMES experiments are reported on a subset of first 300 questions, which limits the scope of the evaluation. This keeps the cross-domain comparison feasible within the thesis time and compute budget.

6.1.3 Models

The experiments are carried out with two language models:

- OpenAI GPT-4.1
- GPT-OSS

GPT-4.1 is included as a strong closed-model reference, while GPT-OSS 120 billion parameter model is used to check if the same results also hold for a smaller, more accessible open model.

Both models are evaluated under the same retrieval comparison design across both datasets, while respecting dataset-specific differences such as the exclusion of query expansion on FRAMES. This allows comparison of the retrieval pipeline under two different generation settings and shows whether the effects of hybrid retrieval, query expansion, multi-hop retrieval, and reranking remain consistent across a closed model and an open model.

6.1.4 Metrics

This chapter reports retrieval-level and answer-level metrics. Retrieval quality is measured with precision, recall, and F1. Answer quality is measured with context precision, faithfulness, and answer correctness. The metric definitions and the document-level evaluation setting are described earlier in Section 4.4.1 and Section 4.4.2. For the standards dataset, retrieved chunks are mapped to source PDF files. For FRAMES, retrieved chunks are mapped to Wikipedia articles.

6.1.5 Evaluation Procedure

All experiments use the implementation framework described in Chapter 5. Each configuration is evaluated separately with both language models. Runtime information and debug output are also recorded for later analysis and for discussing performance cost trade-offs. The retrieval modules, fusion, reranking, query expansion, and multi-hop logic are described in Sections 5.5.

6.1.6 Scope of Comparison

The evaluation is organized around following comparisons.

- BM25 baseline vs. Hybrid baseline
- BM25 baseline vs. BM25 with Query expansion
- Hybrid baseline vs. Hybrid with Query expansion
- BM25 baseline vs. BM25 with Multi-hop retrieval
- Hybrid baseline vs. Hybrid with Multi-hop retrieval

- Hybrid baseline vs. Hybrid with reranking

These comparisons are kept separate so that the contribution of each retrieval component can be examined against the corresponding baseline.

6.2 Results

This section presents the results for the evaluated configurations on the PDF dataset and on FRAMES. The analysis is grouped by dataset and model.

6.2.1 PDF Dataset with GPT-4.1

The following results compare the evaluated BM25 and hybrid configurations on the standards-based PDF dataset with GPT-4.1. They are used to examine how hybrid retrieval, query expansion, multi-hop retrieval, and reranking affect performance in the main domain-specific setting.

BM25 and Hybrid Baseline Comparison

Following tables compare the BM25 baseline and the hybrid baseline on the PDF dataset with GPT-4.1. The Table 6.1 reports retriever precision, recall, and F1. The Table 6.2 reports context precision, faithfulness, and answer correctness. The comparison is intended to show the effect of adding dense retrieval and rank fusion to the basic BM25 pipeline.

Table 6.1.: Retriever performance on the PDF dataset with GPT-4.1

Configuration	Precision	Recall	F1
BM25 baseline	0.627	0.810	0.678
Hybrid baseline	0.748	0.980	0.814

Table 6.2.: Generation performance on the PDF dataset with GPT-4.1

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.610	0.607	0.629
Hybrid baseline	0.724	0.694	0.680

The hybrid baseline performed better than the BM25 baseline on all reported metrics. The largest gains appeared in retriever recall and retriever F1, with recall increasing from 0.810 to 0.980 and F1 from 0.678 to 0.814. The precision of the retriever also improved from 0.627 to 0.748. These results showed that adding dense retrieval and rank fusion improved the system's ability to retrieve relevant source documents from domain-specific PDFs. The answer-level metrics also improved. For example with context precision increased from 0.610 to 0.724, faithfulness from 0.607 to 0.694, and answer correctness from 0.629 to 0.680. This suggests that the stronger retrieval quality in the hybrid baseline also supports better final answers.

Effect of Query Expansion

Table 6.3 and Table 6.4 compare the baseline and query expansion settings for BM25 and hybrid retrieval on the PDF dataset with GPT-4.1.

Table 6.3.: Retriever performance with query expansion on the PDF dataset with GPT-4.1

Configuration	Precision	Recall	F1
BM25 baseline	0.627	0.810	0.678
BM25 with query expansion	0.752	0.970	0.813
Hybrid baseline	0.748	0.980	0.814
Hybrid with query expansion	0.810	0.990	0.861

Table 6.4.: Generation performance with query expansion on the PDF dataset with GPT-4.1

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.610	0.607	0.629
BM25 with query expansion	0.790	0.708	0.725
Hybrid baseline	0.724	0.694	0.680
Hybrid with query expansion	0.776	0.696	0.725

Query expansion improved both pipelines on the PDF dataset, but the effect differs between BM25 and hybrid retrieval. For BM25, query expansion increased retriever precision from 0.627 to 0.752, recall from 0.810 to 0.970, and F1 from 0.678 to 0.813. The answer-level metrics also improved clearly, with context precision rising from 0.610 to 0.790, faithfulness from 0.607 to 0.708, and answer correctness from 0.629 to 0.725. This indicates that query expansion helped BM25 find relevant terms that original query alone did not match well.

The hybrid pipeline also benefited from query expansion, but the gains were smaller on the answer side. Retriever precision increased from 0.748 to 0.810, recall from 0.980 to 0.990, and F1 from 0.814 to 0.861, while answer correctness increased from 0.680 to 0.725. However, faithfulness changed only slightly from 0.694 to 0.696, and context precision increased more moderately from 0.724 to 0.776. This suggests that query expansion is more beneficial for BM25, whereas the hybrid baseline already captures part of the semantic relation between the query and the document through dense retrieval and fusion.

The following example shows the benefit of query expansion on the PDF dataset.

Question: *During the H point procedure, if the three-dimensional H-point machine does not slide rearward naturally, what specific manual action is prescribed?*

Target source: `AIS_021.pdf`

BM25 baseline:

- Retrieved files = `AIS_127.pdf`
- Retriever precision = 0.0
- Retriever recall = 0.0
- Retriever F1 = 0.0

BM25 with query expansion:

- Expanded query used `("h point" OR "hip point")`
- Retrieved files = `AIS_021.pdf`
- Retriever precision = 1.0
- Retriever recall = 1.0
- Retriever F1 = 1.0

The baseline query missed the relevant document, whereas the expanded query added the synonym *hip point*, which matched the terminology used in the correct source and enabled the retriever to find the target file directly.

Effect of Multi-hop Retrieval

Table 6.5 and Table 6.6 compare the baseline and multi-hop settings for BM25 and hybrid retrieval on the PDF dataset with GPT-4.1.

Multi-hop retrieval showed a mixed effect on the PDF dataset. Overall, it slightly increased BM25 recall from 0.810 to 0.820, but precision dropped from 0.627 to 0.574, which lowered F1 from 0.678 to 0.638. For the hybrid pipeline, recall

Table 6.5.: Retriever performance with multi-hop retrieval on the PDF dataset with GPT-4.1

Configuration	Precision	Recall	F1
BM25 baseline	0.627	0.810	0.678
BM25 with multi-hop	0.574	0.820	0.638
Hybrid baseline	0.748	0.980	0.814
Hybrid with multi-hop	0.699	0.980	0.773

Table 6.6.: Generation performance with multi-hop retrieval on the PDF dataset with GPT-4.1

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.610	0.607	0.629
BM25 with multi-hop	0.615	0.634	0.645
Hybrid baseline	0.724	0.694	0.680
Hybrid with multi-hop	0.725	0.701	0.738

remained at 0.980, while precision decreased from 0.748 to 0.699 and F1 from 0.814 to 0.773. This means that the second retrieval step did not consistently find new relevant source documents, but it often added extra non-target documents.

The following example shows the typical effect of multi-hop retrieval on the PDF dataset.

Question: *Which condition must exist for a vehicle to be dispatched without a human onboard?*

Target source: `Automotive_SPICE_PAM_v40.pdf`

Hybrid baseline:

- Retrieved PDFs = `SAE_J3016_202104.pdf`
- Retriever precision = 1.0
- Retriever recall = 1.0
- Retriever F1 = 1.0

Hybrid with multi-hop:

- Retrieved PDFs = `SAE_J3016_202104.pdf`,`Automotive_SPICE_PAM_v40.pdf`, `AIS_127.pdf`, and `AIS_189.pdf`
- Retriever precision = 0.25
- Retriever recall = 1.0
- Retriever F1 = 0.40

The relevant source document was already retrieved in the hybrid baseline run, but the additional hop introduced extra documents that were not part of the target evidence set. As a result, retrieval precision and F1 decreased even though the correct source remained in the retrieved set.

Effect of Reranking

Tables 6.7 and 6.8 present the retriever and generation performance of the hybrid baseline and hybrid reranking configurations on the PDF dataset with GPT-4.1.

Table 6.7.: Retriever performance with reranking on the PDF dataset with GPT-4.1

Configuration	Precision	Recall	F1
Hybrid baseline	0.748	0.980	0.814
Hybrid with reranking	0.813	0.980	0.863

Table 6.8.: Generation performance with reranking on the PDF dataset with GPT-4.1

Configuration	Context Precision	Faithfulness	Answer Correctness
Hybrid baseline	0.724	0.694	0.680
Hybrid with reranking	0.785	0.714	0.703

Reranking improved the hybrid pipeline on the PDF dataset mainly by improving the final ordering of retrieved candidates rather than by finding additional relevant documents. This can be seen in the retrieval metrics: recall remained unchanged at 0.980, while precision increased from 0.748 to 0.813 and F1 from 0.814 to 0.863. The answer-level metrics also improved, with context precision rising from 0.724 to 0.785, faithfulness from 0.694 to 0.714, and answer correctness from 0.680 to 0.703. This indicates that reranking makes the top retrieved context cleaner and more relevant, which also supports better final answers.

The following example shows the effect of reranking clearly.

Question: *If machine learning model retraining is triggered after data drift detection, which earlier phase enabled monitoring of that drift?*

Target source: `Automotive_SPICE_PAM_v40.pdf`

Hybrid baseline:

- Retrieved PDFs = `Automotive_SPICE_PAM_v40.pdf`, `SAE_J3016_202104.pdf`
- Retriever precision = 0.5

- Retriever recall = 1.0
- Retriever F1 = 0.667

Hybrid with reranking:

- Retrieved PDFs = `Automotive_SPICE_PAM_v40.pdf`
- Retriever precision = 1.0
- Retriever recall = 1.0
- Retriever F1 = 1.0

Reranking changed the final ordering of the retrieved candidates so that chunks from the target source are ranked higher than less relevant candidates. As a result, the final retrieved set became more precise, which improved precision and F1 while recall remained unchanged.

6.2.2 FRAMES with GPT-4.1

The following results compare the evaluated BM25 and hybrid configurations on FRAMES with GPT-4.1. They are used to examine whether similar retrieval patterns also appear in a cross-domain benchmark setting.

BM25 and Hybrid Baseline Comparison

Tables 6.9 and 6.10 compare the BM25 baseline and the hybrid baseline on FRAMES with GPT-4.1.

Table 6.9.: Retriever performance on FRAMES with GPT-4.1

Configuration	Precision	Recall	F1
BM25 baseline	0.541	0.518	0.500
Hybrid baseline	0.658	0.548	0.564

Table 6.10.: Generation performance on FRAMES with GPT-4.1

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.148	0.557	0.319
Hybrid baseline	0.205	0.614	0.359

The hybrid baseline performed better than the BM25 baseline on FRAMES with GPT-4.1 across both retrieval-level and answer-level metrics. Retriever precision

increased from 0.541 to 0.658, recall from 0.518 to 0.548, and F1 from 0.500 to 0.564. The answer-level metrics also improved, with context precision rising from 0.148 to 0.205, faithfulness from 0.557 to 0.614, and answer correctness from 0.319 to 0.359. This indicates that combining sparse and dense retrieval was also beneficial in the cross-domain setting and not only in the standards-based PDF dataset. However, the absolute scores are clearly lower than on the PDF dataset, which suggests that FRAMES is a more difficult retrieval setting. The benchmark is open-domain, evaluated here on a 300-question subset, and relies on referenced Wikipedia articles rather than a narrow domain corpus. As a result, retrieval was less constrained and the effect of the retrieval strategy appears less strong than in the domain-specific PDF setting.

Effect of Multi-hop Retrieval

Tables 6.11 and 6.12 compare the baseline and multi-hop settings for BM25 and hybrid retrieval on FRAMES with GPT-4.1.

Table 6.11.: Retriever performance with multi-hop retrieval on FRAMES with GPT-4.1

Configuration	Precision	Recall	F1
BM25 baseline	0.541	0.518	0.500
BM25 with multi-hop	0.524	0.613	0.530
Hybrid baseline	0.658	0.548	0.564
Hybrid with multi-hop	0.646	0.644	0.605

Table 6.12.: Generation performance with multi-hop retrieval on FRAMES with GPT-4.1

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.148	0.557	0.319
BM25 with multi-hop	0.190	0.599	0.350
Hybrid baseline	0.205	0.614	0.359
Hybrid with multi-hop	0.199	0.621	0.405

Multi-hop retrieval had a positive effect on FRAMES. For BM25, recall increased from 0.518 to 0.613 and answer correctness from 0.319 to 0.350. For the hybrid pipeline, recall increased from 0.548 to 0.644 and answer correctness from 0.359 to 0.405. This shows that the second retrieval step recovered additional relevant evidence, which improved answer generation overall. At the same time, the retrieved set became broader, so some extra non-target articles were also added, which decreased

precision slightly. However, on FRAMES the gain in relevant evidence was larger than this downside, so the overall results still improved.

This example shows the effect of multi-hop on FRAMES clearly.

Question: *In the style of wrestling performed by former Greek wrestler Mikhail Theodoropoulos, who won the inaugural Olympic medal?*

Target wikipedia articles:

`Greco-Roman wrestling`, `List of World and Olympic Champions in Greco-Roman wrestling`, `Wrestling at the 1960 Summer Olympics - Men's Greco-Roman bantamweight`, `Mikhail Theodoropoulos`

Hybrid baseline:

- Retrieved wikipedia articles = `Mikhail Theodoropoulos`, `Greco-Roman wrestling`, `1912 Summer Olympics`
- Retriever precision = 0.667
- Retriever recall = 0.500
- Retriever F1 = 0.571
- Answer correctness = 0.183

Hybrid with multi-hop:

- Retrieved wikipedia articles = `Mikhail Theodoropoulos`, `Greco-Roman wrestling`, `1912 Summer Olympics`, `2004 Summer Olympics`, `Summer Olympic Games`, `Wrestling at the 2020 Summer Olympics - Men's freestyle 86 kg`
- Retriever precision = 0.333
- Retriever recall = 0.500
- Retriever F1 = 0.400
- Answer correctness = 0.964

In the baseline setting, the retriever used only the original question and retrieved part of the evidence needed for answering. With multi-hop retrieval, the system rewrote the query using additional contextual clues. This led to the retrieval of more related articles and enabled the system to produce the correct answer. However, the broader query also introduced related but non-target evidence into the final retrieved set. As a result, retrieval precision and F1 decreased. Thus, multi-hop substantially improved answer correctness, even though retrieval precision and F1 decreased.

Effect of Reranking

Tables 6.13 and 6.14 compare the hybrid baseline and the hybrid reranking configuration on FRAMES with GPT-4.1.

Table 6.13.: Retriever performance with reranking on FRAMES with GPT-4.1

Configuration	Precision	Recall	F1
Hybrid baseline	0.658	0.548	0.564
Hybrid with reranking	0.623	0.548	0.551

Table 6.14.: Generation performance with reranking on FRAMES with GPT-4.1

Configuration	Context Precision	Faithfulness	Answer Correctness
Hybrid baseline	0.205	0.614	0.359
Hybrid with reranking	0.226	0.605	0.347

Reranking did not improve the hybrid pipeline on FRAMES with GPT-4.1. The results showed that it changed the ordering of the already retrieved candidates, but this reordered ranking did not help the final answer overall. This was visible in the metric pattern: recall remained unchanged at 0.548, which means reranking did not recover additional relevant articles, while precision decreased from 0.658 to 0.623 and F1 from 0.564 to 0.551. At the answer level, faithfulness decreased from 0.614 to 0.605 and answer correctness from 0.359 to 0.347, although context precision increased slightly from 0.205 to 0.226.

Overall, reranking changed the ranking of retrieved candidates, but this did not lead to better overall performance on FRAMES.

This example shows the negative effect of reranking on FRAMES clearly.

Question: *Based on the information available on Wikipedia on August 4, 2024 at 2:42 AM Greenwich Mean Time, which of the following areas, Minamidaitōjima, Nuapada district, or Vostochnaya Niva has the highest population and how many more citizens does it contain than the other two provided cities?*

Target wikipedia articles: `Nuapada district`, `Vostochnaya Niva`, `Minamidaitōjima`

Hybrid baseline:

- Retrieved wikipedia articles = `Vostochnaya Niva`, `Minamidaitōjima`, `Nuapada district`, and `Tokyo`
- Retriever precision = 0.750

- Retriever recall = 1.000
- Retriever F1 = 0.857

Hybrid with reranking:

- Retrieved wikipedia articles = `List of United States cities by population`, `Tokyo`, and `City of London`
- Retriever precision = 0.000
- Retriever recall = 0.000
- Retriever F1 = 0.000

In the baseline setting, the required evidence was retrieved and the question could be answered. With reranking, the relevant articles were pushed out of the final set, which caused retrieval to fail and the answer to become worse.

6.2.3 PDF Dataset with GPT-OSS

The following results compare the evaluated BM25 and hybrid configurations on the standards-based PDF dataset with GPT-OSS. They are used to examine how hybrid retrieval, query expansion, multi-hop retrieval, and reranking affect performance for the open-source model in the domain-specific setting.

BM25 and Hybrid Baseline Comparison

Tables 6.15 and 6.16 compare the BM25 baseline and the hybrid baseline on the PDF dataset with GPT-OSS.

Table 6.15.: Retriever performance on the PDF dataset with GPT-OSS

Configuration	Precision	Recall	F1
BM25 baseline	0.613	0.810	0.667
Hybrid baseline	0.724	0.970	0.793

Table 6.16.: Generation performance on the PDF dataset with GPT-OSS

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.590	0.550	0.504
Hybrid baseline	0.654	0.633	0.620

The hybrid baseline performed better than the BM25 baseline on all reported metrics. Retriever precision increased from 0.613 to 0.724, recall from 0.810 to 0.970, and F1 from 0.667 to 0.793. The answer-level metrics also improved, with context precision rising from 0.591 to 0.654, faithfulness from 0.550 to 0.633, and answer correctness from 0.504 to 0.620. This shows that combining sparse and dense retrieval provided a stronger retrieval basis than BM25 alone on the PDF dataset and that the more reliable retrieved evidence also helped GPT-OSS generate better answers.

Effect of Query Expansion

Tables 6.17 and 6.18 compare the baseline and query expansion settings for BM25 and hybrid retrieval on the PDF dataset with GPT-OSS.

Table 6.17.: Retriever performance with query expansion on the PDF dataset with GPT-OSS

Configuration	Precision	Recall	F1
BM25 baseline	0.613	0.810	0.667
BM25 with query expansion	0.751	0.970	0.812
Hybrid baseline	0.724	0.970	0.793
Hybrid with query expansion	0.766	0.980	0.829

Table 6.18.: Generation performance with query expansion on the PDF dataset with GPT-OSS

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.590	0.550	0.504
BM25 with query expansion	0.733	0.721	0.656
Hybrid baseline	0.654	0.633	0.620
Hybrid with query expansion	0.716	0.735	0.668

Query expansion improved both pipelines on the PDF dataset with GPT-OSS, with a stronger effect for BM25. For BM25, retriever F1 increased from 0.667 to 0.812 and answer correctness from 0.504 to 0.656. For the hybrid pipeline, the gains were smaller, with F1 increasing from 0.793 to 0.829 and answer correctness from 0.620 to 0.668. This shows that query expansion was most beneficial when retrieval depends more directly on lexical matching, while the hybrid pipeline already captured part of the relation through dense retrieval.

The following example shows the effect of query expansion clearly.

Question: *An Adaptive Front-Lighting System is designed for a vehicle market that requires temporary use in territories with opposite traffic directions. What specific user-accessible function must the system provide to avoid dazzling oncoming traffic?*

Target source: `AIS_127.pdf`

BM25 baseline:

- Retrieved PDFs = `AIS_012.pdf`
- Retriever precision = 0.0
- Retriever recall = 0.0
- Retriever F1 = 0.0

BM25 with query expansion:

- Retrieved PDFs = `AIS_127.pdf`
- Retriever precision = 1.0
- Retriever recall = 1.0
- Retriever F1 = 1.0

In the baseline setting, BM25 relied only on the original wording of the question and retrieved a non-target document. With query expansion, the retriever added the terms `adaptive front lighting` and `afs`, as shown in the debug output. This expansion reduced the lexical mismatch between the full term used in the question and the abbreviated form that may appear in the standard. As a result, BM25 retrieved the correct source document.

Effect of Multi-hop Retrieval

Tables 6.19 and 6.20 compare the baseline and multi-hop settings for BM25 and hybrid retrieval on the PDF dataset with GPT-OSS.

Table 6.19.: Retriever performance with multi-hop retrieval on the PDF dataset with GPT-OSS

Configuration	Precision	Recall	F1
BM25 baseline	0.613	0.810	0.667
BM25 with multi-hop	0.615	0.860	0.677
Hybrid baseline	0.724	0.970	0.793
Hybrid with multi-hop	0.704	0.990	0.779

Multi-hop retrieval had a mixed effect on the PDF dataset with GPT-OSS. For BM25, it improved recall from 0.810 to 0.860 and answer correctness from 0.504 to 0.600.

Table 6.20.: Generation performance with multi-hop retrieval on the PDF dataset with GPT-OSS

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.590	0.550	0.504
BM25 with multi-hop	0.613	0.567	0.600
Hybrid baseline	0.654	0.633	0.620
Hybrid with multi-hop	0.646	0.694	0.661

This means that the second hop helped recover relevant documents that were missed in the baseline setting. At the same time, precision changed only slightly and F1 increased only from 0.667 to 0.677, which showed that some of the additional retrieved documents were not relevant.

For the hybrid pipeline, the effect was similar but less favorable at retrieval level. Recall increased from 0.970 to 0.990 and answer correctness improved from 0.620 to 0.661, so the extra hop still helped the model in some cases. However, precision decreased from 0.724 to 0.704 and retriever F1 decreased from 0.793 to 0.779. This indicates that the second hop added more context, but part of that context was irrelevant, which slightly reduced overall retrieval quality.

This example shows the trade-off of multi-hop in the hybrid pipeline.

Question: *To tolerate steering-wheel rim obstruction, the tangent plane through V2 must be declined by at least how many degrees?*

Target source: `AIS_021.pdf`

Hybrid baseline:

- Retrieved PDFs = `AIS_021.pdf`
- Retriever precision = 1.0
- Retriever recall = 1.0
- Retriever F1 = 1.0

Hybrid with multi-hop:

- Retrieved PDFs = `AIS_021.pdf`, `AIS_015.pdf`
- Retriever precision = 0.333
- Retriever recall = 1.0
- Retriever F1 = 0.5

In the baseline setting, the hybrid retriever already returned the correct source document. With multi-hop enabled, later hops introduced additional non-target documents, which reduced retrieval precision and retriever F1. The rewritten query gradually drifted away from the original target concept. This example illustrates that multi-hop increased retrieval noise when the initial retrieval was already sufficient at the document level.

Effect of Reranking

Tables 6.21 and 6.22 compare the hybrid baseline and the hybrid reranking configuration on the PDF dataset with GPT-OSS.

Table 6.21.: Retriever performance with reranking on the PDF dataset with GPT-OSS

Configuration	Precision	Recall	F1
Hybrid baseline	0.724	0.970	0.793
Hybrid with reranking	0.810	0.980	0.859

Table 6.22.: Generation performance with reranking on the PDF dataset with GPT-OSS

Configuration	Context Precision	Faithfulness	Answer Correctness
Hybrid baseline	0.654	0.633	0.620
Hybrid with reranking	0.746	0.645	0.667

Reranking improved the hybrid pipeline on the PDF dataset with GPT-OSS because it made the final retrieved context cleaner without reducing evidence coverage. This can be seen in the retrieval metrics: recall remained almost unchanged, increasing only from 0.970 to 0.980, while precision increased more clearly from 0.724 to 0.810 and F1 from 0.793 to 0.859. The main effect of reranking was therefore not to find many new relevant source documents, but to move the more useful ones higher in the final ranking and reduce the influence of weaker candidates. This also improved the answer-level results, especially context precision and answer correctness, which indicates that GPT-OSS benefits when the top-ranked evidence was more focused and relevant.

The following example shows the effect of reranking in the hybrid pipeline.

Question: *For a vehicle submission where the manufacturer has installed specialized belt routing hardware that alters the direct path between anchor points, what testing principle ensures the applied forces accurately represent real-world loading conditions?*

Target source: `AIS_015.pdf`.

Hybrid baseline:

- Retrieved PDFs = `AIS_012.pdf`, `is15139_2002.pdf`, and `IS15140_2018.pdf`
- Retriever precision = 0.0
- Retriever recall = 0.0
- Retriever F1 = 0.0

Hybrid with reranking:

- Retrieved PDFs = `AIS_015.pdf`, `IS15140_2018.pdf`, and `is15139_2002.pdf`
- Retriever precision = 0.333
- Retriever recall = 1.0
- Retriever F1 = 0.5

In the baseline setting, the hybrid retriever did not return the target source document. With reranking enabled, the correct source document, `AIS_015.pdf`, appeared in the final retrieved set. As a result, retrieval quality improved compared with the baseline. This example shows that reranking helped place the right source document at the top of the list.

6.2.4 FRAMES with GPT-OSS

The following results compare the evaluated BM25 and hybrid configurations on FRAMES with GPT-OSS. They are used to examine how the retrieval pipeline behaves for the open-source model in the cross-domain setting.

BM25 and Hybrid Baseline Comparison

Tables 6.23 and 6.24 compare the BM25 baseline and the hybrid baseline on FRAMES with GPT-OSS.

Table 6.23.: Retriever performance on FRAMES with GPT-OSS

Configuration	Precision	Recall	F1
BM25 baseline	0.541	0.518	0.500
Hybrid baseline	0.442	0.493	0.443

On FRAMES with GPT-OSS, the baseline comparison showed a clear mismatch between retrieval-level and answer-level behavior. The hybrid baseline generated

Table 6.24.: Generation performance on FRAMES with GPT-OSS

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.171	0.182	0.265
Hybrid baseline	0.225	0.243	0.314

better answer-level results, with context precision increasing from 0.171 to 0.225, faithfulness from 0.182 to 0.243, and answer correctness from 0.265 to 0.314. However, the retrieval-level metrics became lower than for BM25, with precision dropping from 0.541 to 0.442, recall from 0.518 to 0.493, and F1 from 0.500 to 0.443. This was a mixed result, but it was not contradictory. In FRAMES, retrieval was evaluated at the level of referenced Wikipedia articles. The hybrid pipeline therefore received a lower retrieval score when it brought in semantically useful but non-target articles, even if those articles still helped the model produce a better answer.

These results showed that FRAMES was harder and less stable for GPT-OSS than the PDF dataset. On this benchmark, answer quality did not depend only on exact recovery of the referenced gold articles. The model can sometimes answer correctly from partially matching or semantically related evidence, which helps explain why the hybrid baseline improved answer quality even though its article-level retrieval metrics were weaker than those of BM25.

Effect of Multi-hop Retrieval

Tables 6.25 and 6.26 compare the baseline and multi-hop settings for BM25 and hybrid retrieval on FRAMES with GPT-OSS.

Table 6.25.: Retriever performance with multi-hop retrieval on FRAMES with GPT-OSS

Configuration	Precision	Recall	F1
BM25 baseline	0.541	0.518	0.500
BM25 with multi-hop	0.515	0.625	0.528
Hybrid baseline	0.442	0.493	0.443
Hybrid with multi-hop	0.368	0.584	0.418

On FRAMES with GPT-OSS, multi-hop retrieval gave mixed results and was less effective at the retrieval level for the hybrid pipeline. For BM25, recall increased from 0.518 to 0.625 and retriever F1 from 0.500 to 0.528, while answer correctness increased slightly from 0.265 to 0.278. This suggests that the second hop helped

Table 6.26.: Generation performance with multi-hop retrieval on FRAMES with GPT-OSS

Configuration	Context Precision	Faithfulness	Answer Correctness
BM25 baseline	0.171	0.182	0.265
BM25 with multi-hop	0.159	0.201	0.278
Hybrid baseline	0.225	0.243	0.314
Hybrid with multi-hop	0.211	0.249	0.353

BM25 recover additional relevant articles, even though precision decreased slightly from 0.541 to 0.515.

For the hybrid pipeline, the effect was less favorable. Although recall increased from 0.493 to 0.584 and answer correctness from 0.314 to 0.353, precision fell from 0.442 to 0.368 and retriever F1 dropped from 0.443 to 0.418.

This indicates that the second hop improved recall but also added a lot of noise, by adding articles that were not relevant or loosely related to the topics which reduced the overall retrieval quality.

The following example shows this limitation for the hybrid pipeline clearly.

Question: *As of August 1 2024, what books has the author of the Harry Potter series written under an alias?*

Target wikipedia articles: `J. K. Rowling`, `Harry Potter`

Hybrid baseline:

- Retrieved wikipedia articles = `J. K. Rowling`, `Harry Potter`, `Harry Potter and the Goblet of Fire`, `Harry Potter and the Philosopher's Stone`, `Iron Flame`
- Retriever precision = 0.400
- Retriever recall = 1.000
- Retriever F1 = 0.571
- Answer correctness = 0.289

Hybrid with multi-hop:

- Retrieved wikipedia articles = `J. K. Rowling`, `Harry Potter`, `Harry Potter and the Goblet of Fire`, `Harry Potter and the Philosopher's Stone`, `Iron Flame`, `Philip K. Dick`
- Retriever precision = 0.333
- Retriever recall = 1.000

- Retriever F1 = 0.500
- Answer correctness = 0.863

Here, the baseline retrieved some relevant articles, but the evidence remained too general and did not contain the specific information required to answer the question. In the multi-hop setting, the query was rewritten more specifically, which retrieved enough evidence to produce a much more complete answer. However, this broader search also introduced irrelevant articles, which lowered retriever precision and F1.

Effect of Reranking

Tables 6.27 and 6.28 compare the hybrid baseline and the hybrid reranking configuration on FRAMES with GPT-OSS.

Table 6.27.: Retriever performance with reranking on FRAMES with GPT-OSS

Configuration	Precision	Recall	F1
Hybrid baseline	0.442	0.493	0.443
Hybrid with reranking	0.597	0.536	0.536

Table 6.28.: Generation performance with reranking on FRAMES with GPT-OSS

Configuration	Context Precision	Faithfulness	Answer Correctness
Hybrid baseline	0.225	0.243	0.314
Hybrid with reranking	0.225	0.229	0.315

On FRAMES with GPT-OSS, reranking improved retrieval performance much more clearly than answer generation. Retriever precision increased from 0.442 to 0.597, recall from 0.493 to 0.536, and F1 from 0.443 to 0.536. This shows that reranking produced a substantially better final ordering of the retrieved articles in the cross-domain setting.

However, the answer-level metrics remained nearly unchanged. Context precision remained at 0.225, answer correctness increased only marginally from 0.314 to 0.315, and faithfulness decreased slightly from 0.243 to 0.229. This indicates that better article-level retrieval did not automatically translate into better answer generation for GPT-OSS on FRAMES.

Overall, reranking made the retrieved evidence set cleaner and more accurate, but the model did not consistently convert that improvement into more faithful or more

correct answers. This again reflects the difficulty of FRAMES, where cross-article reasoning is required and stronger retrieval alone is often not sufficient.

The following example shows that improved retrieval quality does not always lead to improved answer generation.

Question: *As of August 1, 2024, which NFL players were both league MVP and Super Bowl MVP in the same season?*

Target Wikipedia articles: `Associated Press NFL Most Valuable Player Award`, `Super Bowl Most Valuable Player Award`

Hybrid baseline:

- Retriever precision = 0.200
- Recall = 0.500
- F1 = 0.285
- Answer correctness = 0.936

Hybrid with rerank:

- Retriever precision = 1.000
- Recall = 1.000
- F1 = 1.000
- Answer correctness = 0.737

Here, reranking produces a cleaner final ordering of the retrieved articles, which improved retriever precision, recall, and F1. However, the answer becomes less accurate. The answer still did not fully match the question, even though the more relevant articles appeared higher in the ranking.

6.2.5 Cross-Model Comparison

This subsection compares the results of GPT-4.1 and GPT-OSS across the evaluated settings on the PDF dataset and on FRAMES. The goal is to examine whether the same retrieval trends appear across the two models and where the two models respond differently to the tested retrieval settings.

Overall, both models show the same broad pattern on the PDF dataset. The hybrid pipeline is stronger than the BM25 baseline, query expansion improves both pipelines, and reranking improves the hybrid setup. This indicates that the main

retrieval effects are not specific to one model, but reflect stable properties of the retrieval pipeline itself.

At the same time, GPT-4.1 achieves higher answer-level results than GPT-OSS across nearly all PDF configurations. The retrieval-level metrics are often closer between the two models than the answer-level metrics, which suggests that part of the performance gap comes not from retrieval alone, but from how effectively each model uses the retrieved evidence.

The difference becomes clearer on FRAMES. With GPT-4.1, the hybrid baseline improves both retrieval-level and answer-level metrics over BM25. With GPT-OSS, however, the hybrid baseline improves answer-level metrics but gives lower retrieval-level scores than BM25. This shows that in the cross-domain benchmark, the relation between retrieval quality and answer quality becomes less stable, especially for the open-source model.

Multi-hop retrieval highlights the same distinction. On FRAMES, multi-hop improves answer correctness for both models, but the metric pattern is more unstable for GPT-OSS. In particular, hybrid multi-hop with GPT-OSS increases recall and answer correctness while precision and F1 decrease more strongly than with GPT-4.1. This suggests that broader retrieval helps both models recover more evidence, but the added context is noisier and harder for GPT-OSS to use reliably.

Reranking shows a similar contrast. On the PDF dataset, reranking helps the hybrid pipeline for both models. On FRAMES, however, reranking does not improve the hybrid pipeline overall for GPT-4.1, while for GPT-OSS it improves retrieval-level metrics more clearly than answer-level metrics. This again points to a retrieval-generation gap in the open-source setting.

Taken together, the cross-model comparison shows that the retrieval components behave consistently at a high level, but their final impact depends on the model. In the PDF setting, both models benefit from stronger retrieval, although GPT-4.1 makes more effective use of the retrieved evidence. In FRAMES, the gap becomes larger, and improvements in retrieval are less reliably converted into better answers.

6.3 Discussion

6.3.1 Main Findings

The evaluation shows that retrieval strategy has a clear effect on both retrieval quality and answer quality. In the standards-based PDF setting, the hybrid pipeline is the strongest baseline for both models, query expansion is especially beneficial for BM25, and reranking improves the hybrid setup by making the final retrieved context more precise. These results indicate that in a narrow domain, retrieval improvements are reflected relatively directly in answer quality.

Multi-hop retrieval produces the clearest trade-off in the evaluation. On the PDF dataset, it often broadens the retrieved context without adding much new relevant evidence. On FRAMES, it is more useful because many questions require evidence from multiple articles, but the added coverage also introduces more noise. Overall, the benefit of each retrieval component depends on both the dataset structure and the model's ability to use the retrieved evidence effectively.

6.3.2 Effect of Query Expansion

Query expansion is beneficial on the PDF dataset for both models, but its effect is stronger for BM25 than for the hybrid pipeline. This is because BM25 depends directly on lexical overlap between the question and the document text. When domain-specific abbreviations, synonyms, or alternative terms are added to the query, BM25 can recover relevant standards content that would otherwise be missed.

The hybrid pipeline also benefits from query expansion, but less strongly, because dense retrieval already captures part of the semantic relation between the question and the document even without explicit term expansion. The observed benefit of query expansion is domain-specific to the standards corpus. It was not evaluated on FRAMES and therefore does not generalize to all datasets.

6.3.3 Effect of Multi-hop Retrieval

Multi-hop retrieval produces the clearest trade-off in the evaluation. The main benefit is improved evidence coverage, though this often reduces retrieval precision. In the PDF dataset, many questions can already be answered using one main standards document, since most questions are derived from a single source.

In such cases, the second hop often keeps the correct source but adds extra related documents that are not actually needed for the answer. This explains why recall may stay similar or increase slightly while precision, and sometimes F1, decrease.

This pattern is visible in the example discussed earlier, where the correct PDF source remains retrieved but additional non-target files reduce precision without improving the actual evidence coverage.

On FRAMES, the role of multi-hop is different. Many questions require connecting evidence across multiple referenced Wikipedia articles, so a second retrieval step is more useful there than on the PDF dataset. However, the second hop also brings in more loosely related articles, which lowers precision and makes the retrieved set noisier. This effect is especially visible for GPT-OSS, where hybrid multi-hop increases answer correctness but lowers precision and F1. Multi-hop is therefore valuable for coverage in the cross-domain setting, but its benefit remains unstable because the retrieved evidence chain is not always complete or clean.

6.3.4 Effect of Reranking

Reranking is clearly beneficial on the PDF dataset. Its main effect is not to retrieve many new relevant documents, but to improve the final ordering of the already retrieved candidates. This makes the top-ranked evidence more focused and relevant, which raises retrieval precision and F1 and also improves answer-level metrics.

This pattern is visible in the earlier data-drift example, where reranking keeps the correct source but pushes non-target material lower in the final ranking, thereby improving precision without changing recall.

On FRAMES, reranking is less consistent. With GPT-4.1, it does not improve the hybrid pipeline overall. With GPT-OSS, it improves retrieval-level metrics clearly, but answer-level metrics remain almost unchanged. This shows that cleaner article-level retrieval does not automatically lead to better final answers in the cross-domain setting.

6.3.5 100-QA and FRAMES Dataset Interpretation

The comparison across datasets shows that the retrieval components behave differently in the domain-specific and cross-domain settings. On the PDF dataset, the retrieval pipeline behaves more predictably: hybrid retrieval is consistently stronger

than BM25, query expansion helps lexical retrieval, and reranking improves the hybrid setup. Because the standards corpus is narrower and more structured than FRAMES, retrieval improvements are reflected more directly in answer quality.

FRAMES is more difficult because it is open-domain and uses Wikipedia-based sources instead of a narrow corpus. Here, it was evaluated on first 300-question subset. Many questions require connecting multiple pieces of evidence across different articles. As a result, retrieval becomes less stable, multi-hop becomes more useful for coverage, and reranking becomes less reliable as a way of improving final answers.

Overall, the results show that retrieval performance depends a lot on the dataset structure. In the narrower PDF setting, better retrieval usually leads more directly to better answers. In FRAMES, the broader and more complex evidence makes retrieval noisier, so retrieval does not always lead to better final answers.

Failure Analysis

The error cases in the evaluation do not point to a single isolated weakness. Instead, they reveal several recurring failure patterns across datasets, retrieval settings, and models.

Noise introduced by multi-hop retrieval: The first recurring pattern is additional noise introduced by multi-hop retrieval. On the PDF dataset, the second hop often adds standards documents that are related to the topic but not actually required for the answer. As a result, retrieval precision decreases even when the correct source has already been found in the first hop. In such cases, the second hop broadens the context without improving the actual evidence needed for answering. This indicates that the current multi-hop strategy is sometimes too broad for questions that can already be resolved from a single main document.

Incomplete evidence for multi-step questions: The second recurring pattern is missing links in multi-step evidence chains. This is especially important on FRAMES, where many questions require evidence from more than one article. When the retriever finds only part of the required evidence chain, the model may still lack a necessary intermediate or supporting article, even if one relevant source has already been retrieved. In such cases, answer generation often fails because the available context is incomplete. Multi-hop retrieval can reduce this problem by expanding the

search beyond the first retrieved results, but it does not guarantee that all required linked evidence will be recovered.

Improved retrieval without better answers: The third recurring pattern is improved ranking quality without comparable answer improvement. This is most clearly visible for reranking on FRAMES with GPT-OSS. There, reranking improves article-level precision, recall, and F1, but answer correctness remains almost unchanged and faithfulness decreases slightly. In this case, the reranker produces a cleaner and more accurate final ordering of the already retrieved articles, but the model does not consistently convert that improvement into better final answers. This pattern points to a retrieval-generation gap: better ranked evidence does not automatically lead to corresponding gains in answer quality, especially in more difficult cross-document reasoning settings.

Implications for future improvement: These failure patterns suggest that future improvements should focus on making retrieval more selective and more reasoning-aware. In particular, multi-hop retrieval could be triggered only when the first-hop evidence appears incomplete or when the question clearly requires multiple evidence steps. Reranking could also be adapted to prioritize combinations of evidence across documents rather than only local passage relevance.

6.3.6 Runtime Trade-Offs

Besides retrieval quality and answer quality, the evaluated configurations also differ in computational cost. To examine this aspect, the average retrieval time per question is compared across datasets, models, and retrieval settings. The reported values are computed from the `retrieval_time_sec` values recorded for each evaluated question. This comparison is intended to show how much additional runtime is introduced by query expansion, multi-hop retrieval, and reranking relative to the corresponding baselines. The reported values are average per-question retrieval times, computed from the `retrieval_time_sec` values recorded for each evaluated question.

Tables 6.29 and 6.30 show the same overall pattern. Query expansion adds almost no extra retrieval time. Multi-hop retrieval is the most expensive setting in both models. Reranking also increases retrieval time, but less than multi-hop retrieval.

Table 6.29.: Average retrieval time per question (seconds) for GPT-4.1.

Dataset	Configuration	Avg. retrieval time (s)
PDF	BM25 baseline	0.016
PDF	BM25 + query expansion	0.018
PDF	BM25 + multi-hop	5.194
PDF	Hybrid baseline	0.105
PDF	Hybrid + query expansion	0.088
PDF	Hybrid + multi-hop	4.016
PDF	Hybrid + reranking	3.523
FRAMES	BM25 baseline	0.914
FRAMES	BM25 + multi-hop	10.169
FRAMES	Hybrid baseline	1.055
FRAMES	Hybrid + multi-hop	11.298
FRAMES	Hybrid + reranking	6.511

Table 6.30.: Average retrieval time per question (seconds) for GPT-OSS.

Dataset	Configuration	Avg. retrieval time (s)
PDF	BM25 baseline	0.018
PDF	BM25 + query expansion	0.019
PDF	BM25 + multi-hop	11.351
PDF	Hybrid baseline	0.192
PDF	Hybrid + query expansion	0.190
PDF	Hybrid + multi-hop	15.548
PDF	Hybrid + reranking	2.274
FRAMES	BM25 baseline	0.798
FRAMES	BM25 + multi-hop	22.233
FRAMES	Hybrid baseline	1.702
FRAMES	Hybrid + multi-hop	31.774
FRAMES	Hybrid + reranking	5.150

The main difference is that the absolute retrieval times are higher for GPT-OSS than for GPT-4.1.

On the PDF dataset, baseline retrieval is very fast for both models. Multi-hop retrieval increases the average retrieval time from less than 0.2 seconds to 4.016-5.194 seconds for GPT-4.1 and to 11.350-15.548 seconds for GPT-OSS. On FRAMES, retrieval is slower overall. In this setting, multi-hop retrieval increases the average retrieval time to 10.169-11.300 seconds for GPT-4.1 and to 22.233-31.773 seconds for GPT-OSS.

Generation time was also recorded. For GPT-4.1, generation time stays relatively stable across configurations. For GPT-OSS, generation time is generally higher and varies more, especially in the hybrid multi-hop setting. Because GPT-OSS has fewer parameters than GPT-4.1, it may handle more complex retrieved context less efficiently, leading to longer and less consistent generation times. However, the largest latency differences still come from retrieval, not from generation.

Overall, the runtime results show a clear trade-off. Query expansion has almost no cost. Reranking adds noticeable latency. Multi-hop retrieval has the highest computational cost.

6.3.7 Statistical Significance Analysis

Table 6.31.: Wilcoxon signed-rank test results for paired retriever F1 comparisons on the PDF dataset.

Model	Comparison	Config. 1 F1	Config. 2 F1	p-value	Significant?	Better system
GPT-4.1	BM25 vs Hybrid	0.680	0.810	< 0.001	Yes	Hybrid
GPT-4.1	BM25+QE vs Hybrid+QE	0.810	0.860	0.002	Yes	Hybrid+QE
GPT-4.1	BM25+MH vs Hybrid+MH	0.640	0.770	< 0.001	Yes	Hybrid+MH
GPT-4.1	Hybrid vs Hybrid+Rerank	0.810	0.860	0.006	Yes	Hybrid+Rerank
GPT-OSS	BM25 vs Hybrid	0.667	0.793	< 0.001	Yes	Hybrid
GPT-OSS	BM25+QE vs Hybrid+QE	0.811	0.830	0.167	No	None
GPT-OSS	BM25+MH vs Hybrid+MH	0.677	0.779	< 0.001	Yes	Hybrid+MH
GPT-OSS	Hybrid vs Hybrid+Rerank	0.793	0.859	0.001	Yes	Hybrid+Rerank

Table 6.32.: Wilcoxon signed-rank test results for paired retriever F1 comparisons on the FRAMES dataset.

Model	Comparison	Config. 1 F1	Config. 2 F1	p-value	Significant?	Better system
GPT-4.1	BM25 vs Hybrid	0.500	0.560	< 0.001	Yes	Hybrid
GPT-4.1	BM25+MH vs Hybrid+MH	0.530	0.605	< 0.001	Yes	Hybrid+MH
GPT-4.1	Hybrid vs Hybrid+Rerank	0.560	0.551	0.809	No	None
GPT-OSS	BM25 vs Hybrid	0.500	0.443	1.000	No	None
GPT-OSS	BM25+MH vs Hybrid+MH	0.528	0.418	1.000	No	None
GPT-OSS	Hybrid vs Hybrid+Rerank	0.443	0.536	< 0.001	Yes	Hybrid+Rerank

To check whether the observed retriever F1 differences are statistically meaningful, this thesis applies the Wilcoxon signed-rank test [81] to paired per-question retriever F1 scores. Tables 6.31 and 6.32 show that for each comparison, the test examines whether the difference between the first and second configuration is likely to be real or could have occurred by chance. In the tables, the Comparison column shows the two compared configurations, and the Significance column indicates whether the result is statistically significant. The p-value shows the strength of the statistical evidence, and a result is treated as significant when the p-value is below 0.05. In this subsection, a significant result means that the second configuration performs better than the first one. The tables therefore show which retriever F1 differences are statistically supported. In tables, None indicates that no statistically significant evidence was found that the second configuration outperformed the first. For each question, the retriever F1 values of the two compared configurations were paired, and the difference between the first and second configuration was calculated.

6.4 Evaluation Summary

6.4.1 Answers to the Research Questions

The following section answers the research questions described in section 1.2.

Answer to RQ1: The RQ1 is answered by the BM25 and Hybrid Baseline Comparison results on the PDF dataset in Sections 6.2.1 and 6.2.3. Across both GPT-4.1 and GPT-OSS, the hybrid pipeline performs better than the BM25 baseline on both retrieval-level and answer-level metrics. This shows that, in automotive standards question answering, hybrid lexical-semantic retrieval is more effective than lexical retrieval alone. On the PDF dataset, the hybrid baseline improves retriever F1 relative to BM25 by about 20.1% for GPT-4.1 and 18.9% for GPT-OSS.

Answer to RQ2: The RQ2 is answered by the query expansion, multi-hop retrieval, and reranking results on the PDF dataset for both GPT-4.1 and GPT-OSS. Query expansion improves both pipelines, with the strongest benefit for BM25. Reranking improves the hybrid pipeline by making the final retrieved context more precise and by improving answer-level metrics. Multi-hop retrieval has a mixed effect in the PDF setting: it can improve evidence coverage and answer correctness in some cases, but it often introduces additional non-target documents and increases

runtime substantially. Overall, the results show that retrieval enhancements can improve evidence quality and final answer quality, but their usefulness depends on the enhancement type. Among the evaluated enhancements, reranking gives the clearest precision-oriented gain in the PDF setting, improving hybrid retriever F1 by about 6% with GPT-4.1 and about 8% with GPT-OSS. Query expansion gives the strongest relative benefit for BM25, improving BM25 retriever F1 by about 20% with GPT-4.1 and about 22% with GPT-OSS on the PDF dataset.

Answer to RQ3: The RQ3 is answered by the FRAMES results for GPT-4.1 and GPT-OSS and by the dataset interpretation in Section 6.3.5. The results show that the main retrieval patterns transfer only partially from the automotive standards dataset to FRAMES. Hybrid retrieval remains competitive and multi-hop becomes more useful for evidence coverage, but the results are less stable than in the PDF setting. In FRAMES, better retrieval does not always lead to better answers because many questions require evidence from multiple articles.

Answer to RQ4: The RQ4 is answered by the cross-model comparison in Section 6.2.5. The results show that the main retrieval trends are broadly consistent across GPT-4.1 and GPT-OSS. In particular, the hybrid baseline is stronger than BM25 on the PDF dataset, query expansion is helpful, and reranking improves the hybrid pipeline in the domain-specific setting. However, GPT-4.1 achieves stronger answer-level results overall, and the relation between retrieval quality and answer quality is less stable for GPT-OSS, especially on FRAMES.

This chapter evaluated BM25-based and hybrid retrieval pipelines on the 100-QA PDF dataset and on a 300-question subset of FRAMES with GPT-4.1 and GPT-OSS. The results show that retrieval performance depends strongly on dataset structure. In the PDF setting, improvements in retrieval are more directly reflected in answer quality. In FRAMES, better retrieval does not always lead to better answers because many questions require evidence from multiple articles.

Overall, the hybrid pipeline provides the strongest baseline. On the PDF dataset, query expansion is most helpful for BM25, reranking improves retrieval precision, and multi-hop retrieval increases evidence coverage but also has a high runtime cost.

7 Summary

7.1 Summary of the Thesis

This thesis studied RAG for question answering over automotive product safety standards. The goal was to examine how retrieval design affects retrieval quality and final answer quality. For this purpose, the thesis compared a BM25 baseline with a hybrid retrieval pipeline that combines lexical and dense retrieval. It also evaluated query expansion, multi-hop retrieval, and reranking.

The evaluation was carried out on a domain-specific 100-question dataset built from automotive standards PDFs and on a 300-question subset of FRAMES. The results showed that the hybrid pipeline gave good overall performance across the evaluated settings. Query expansion was especially helpful for BM25 on the PDF dataset, reranking improved the precision of the retrieved context, and multi-hop retrieval increased evidence coverage but introduced a clear runtime trade-off. Overall, the results showed that the usefulness of retrieval enhancements depends on the dataset structure and the type of evidence required by the question.

7.2 Limitations

This thesis has several limitations. First, the domain-specific evaluation uses a 100-question dataset and the FRAMES evaluation is limited to a 300-question subset. Second, only two generator models were evaluated, so the findings should not be interpreted as covering all model families. Third, the multi-hop setting uses a prompt-based answerability judge and controlled query rewriting rather than more advanced agentic retrieval strategies. Finally, the PDF pipeline is text-based and does not fully address complex layout elements such as tables, figures, or cross-reference structure.

7.3 Future Work

Several directions can extend this work. A more selective multi-hop strategy could reduce unnecessary extra retrieval steps and lower runtime. Reranking could also be improved for questions that require combining evidence across multiple documents instead of focusing only on local passage relevance. Future work could further evaluate the pipeline on larger datasets, additional technical domains, and more language models. Another useful direction would be to improve document processing for complex PDFs, especially for tables, figures, and other structured content that are only partially preserved in a text-only pipeline.

7.4 Conclusion

This thesis showed that retrieval design matters for retrieval-augmented question answering. The results suggest that no single retrieval enhancement is best in all settings. Instead, effective RAG design depends on the dataset, the retrieval method, and the type of evidence required by the question.

Bibliography

[1] Sallam Abualhaija, Chetan Arora, Amin Sleimi, and Lionel C. Briand. Automated question answering for improved understanding of compliance requirements: A multi-document study. In *2022 IEEE 30th International Requirements Engineering Conference (RE)*, pages 39–50, 2022. doi: 10.1109/RE54965.2022.00011.

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=hSyW5go0v8.

[3] Ahmet Yasin Aytar, Kemal Kilic, and Kamer Kaya. A retrieval-augmented generation framework for academic literature navigation in data science, 2024. URL https://arxiv.org/abs/2412.15404.

[4] Hiteshwar Kumar Azad and Akshay Deepak. Query expansion techniques for information retrieval: A survey. *Information Processing & Management*, 56(5): 1698–1735, 2019. ISSN 0306-4573. doi: https://doi.org/10.1016/j.ipm.2019.05.009. URL https://www.sciencedirect.com/science/article/pii/S0306457318305466.

[5] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI*, CAIN '24, page 194–199, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705915. doi: 10.1145/3644815.3644945. URL https://doi.org/10.1145/3644815.3644945.

[6] Hannah Bast and Claudius Korzen. A benchmark and evaluation for text extraction from pdf. In *2017 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*, pages 1–10, 2017. doi: 10.1109/JCDL.2017.7991564.

[7] J. Bhogal, A. Macfarlane, and P. Smith. A review of ontology based query expansion. *Inf. Process. Manage.*, 43(4):866–886, July 2007. ISSN 0306-

4573. doi: 10.1016/j.ipm.2006.09.003. URL https://doi.org/10.1016/j.ipm.2006.09.003.

[8] Andrew Brown, Muhammad Roman, and Barry Devereux. A systematic literature review of retrieval-augmented generation: Techniques, metrics, and challenges, 2025. URL https://arxiv.org/abs/2508.06401.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

[10] Sebastian Bruch, Siyu Gai, and Amir Ingber. An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, 42(1):1–35, August 2023. ISSN 1558-2868. doi: 10.1145/3596512. URL http://dx.doi.org/10.1145/3596512.

[11] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning*, ICML '05, page 89–96, New York, NY, USA, 2005. Association for Computing Machinery. ISBN 1595931805. doi: 10.1145/1102351.1102363. URL https://doi.org/10.1145/1102351.1102363.

[12] Miao Chen, Xinjiang Lu, Tong Xu, Yanyan Li, Zhou Jingbo, Dejing Dou, and Hui Xiong. Towards table-to-text generation with pretrained language model: A table structure understanding and text deliberating approach. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8199–8210, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.562. URL https://aclanthology.org/2022.emnlp-main.562/.

[13] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In

Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '09, page 758–759, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605584836. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114.

[14] Alessio Di Sandro, Sahar Kokaly, Rick Salay, and Marsha Chechik. Querying automotive system models and safety artifacts with mmint and viatra. In *Proceedings of the 22nd International Conference on Model Driven Engineering Languages and Systems Companion*, MODELS '19 Companion, page 2–11. IEEE Press, 2021. ISBN 9781728151250. doi: 10.1109/MODELS-C.2019.00008. URL https://doi.org/10.1109/MODELS-C.2019.00008.

[15] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazare, Maria Lomeli, Lucas Hosseini, and Herve Jegou. The Faiss Library . *IEEE Transactions on Big Data*, 12(02):346–361, April 2026. ISSN 2332-7790. doi: 10.1109/TBDATA.2025.3618474. URL https://doi.ieeecomputersociety.org/10.1109/TBDATA.2025.3618474.

[16] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2025. URL https://arxiv.org/abs/2404.16130.

[17] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In Nikolaos Aletras and Orphee De Clercq, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.eacl-demo.16. URL https://aclanthology.org/2024.eacl-demo.16/.

[18] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, page 6491–6501, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704901. doi: 10.1145/3637528.3671470. URL https://doi.org/10.1145/3637528.3671470.

[19] Yair Feldman and Ran El-Yaniv. Multi-hop paragraph retrieval for open-domain question answering. In Anna Korhonen, David Traum, and Lluís Màrquez,

editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2296–2309, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1222. URL https://aclanthology.org/P19-1222/.

[20] G. W. Furnas, T. K. Landauer, L. M. Gomez, and S. T. Dumais. The vocabulary problem in human-system communication. *Commun. ACM*, 30(11):964–971, November 1987. ISSN 0001-0782. doi: 10.1145/32206.32212. URL https://doi.org/10.1145/32206.32212.

[21] Shailja Gupta, Rajesh Ranjan, and Surya Narayan Singh. A comprehensive survey of retrieval-augmented generation (rag): Evolution, current landscape and future directions, 2024. URL https://arxiv.org/abs/2410.12837.

[22] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3929–3938. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/guu20a.html.

[23] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, Xianfeng Tang, Qi He, Zhigang Hua, Bo Long, Tong Zhao, Neil Shah, Amin Javari, Yinglong Xia, and Jiliang Tang. Retrieval-augmented generation with graphs (graphrag), 2025. URL https://arxiv.org/abs/2501.00309.

[24] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4320–4333, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.398. URL https://aclanthology.org/2020.acl-main.398/.

[25] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. URL https://arxiv.org/abs/2203.15556.

[26] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. GRAG: Graph retrieval-augmented generation. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4145–4157, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.232. URL https://aclanthology.org/2025.findings-naacl.232/.

[27] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), January 2025. ISSN 1046-8188. doi: 10.1145/3703155. URL https://doi.org/10.1145/3703155.

[28] Xinhao Huang, Zhibo Ren, Yipeng Yu, Ying Zhou, Zulong Chen, and Zeyi Wen. SEAL: Structure and element aware learning improves long structured document retrieval. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 8526–8536, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.429. URL https://aclanthology.org/2025.emnlp-main.429/.

[29] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023. URL http://jmlr.org/papers/v24/23-0037.html.

[30] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Trans. Inf. Syst.*, 20(4):422–446, October 2002. ISSN 1046-8188. doi: 10.1145/582415.582418. URL https://doi.org/10.1145/582415.582418.

[31] Xingyu Ji, Parker Glenn, Aditya G. Parameswaran, and Madelon Hulsebos. Target: Benchmarking table retrieval for generative tasks, 2025. URL https://arxiv.org/abs/2505.11545.

[32] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

[33] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.550. URL https://aclanthology.org/2020.emnlp-main.550/.

[34] Ayman Asad Khan, Md Toufique Hasan, Kai-Kristian Kemell, Jussi Rasku, and Pekka Abrahamsson. Developing retrieval augmented generation (rag) based llm systems from pdfs: An experience report. *ArXiv*, abs/2410.15944, 2024. URL https://api.semanticscholar.org/CorpusID:273502291.

[35] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '20, page 39–48, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450380164. doi: 10.1145/3397271.3401075. URL https://doi.org/10.1145/3397271.3401075.

[36] Claudiu Vasile Kifor and Aurelian Popescu. Automotive cybersecurity: A survey on frameworks, standards, and testing and monitoring technologies. *Sensors*, 24(18), 2024. ISSN 1424-8220. doi: 10.3390/s24186139. URL https://www.mdpi.com/1424-8220/24/18/6139.

[37] Jaewoong Kim, Minseok Hur, and Moohong Min. *From RAG to QA-RAG: Integrating Generative AI for Pharmaceutical Regulatory Compliance Process*, page 1293–1295. Association for Computing Machinery, New York, NY, USA, 2025. ISBN 9798400706295. URL https://doi.org/10.1145/3672608.3707749.

[38] Philip Koopman. Practical experience report: Automotive safety practices vs. accepted principles. In Barbara Gallina, Amund Skavhaug, and Friedemann Bitsch, editors, *Computer Safety, Reliability, and Security*, pages 3–11, Cham, 2018. Springer International Publishing. ISBN 978-3-319-99130-6.

[39] Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational*

Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 4745–4759, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.243. URL https://aclanthology.org/2025.naacl-long.243/.

[40] Saar Kuzi, Mingyang Zhang, Cheng Li, Michael Bendersky, and Marc Najork. Leveraging semantic and lexical matching to improve the recall of document retrieval systems: A hybrid approach, 2020. URL https://arxiv.org/abs/2010.01195.

[41] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.703. URL https://aclanthology.org/2020.acl-main.703/.

[42] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.

[43] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. Retrieval augmented generation or long-context LLMs? a comprehensive study and hybrid approach. In Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 881–893, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.66. URL https://aclanthology.org/2024.emnlp-industry.66/.

[44] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.229. URL https://aclanthology.org/2022.acl-long.229/.

[45] Fei Liu, Zejun Kang, and Xing Han. Optimizing rag techniques for automotive industry pdf chatbots: A case study with locally deployed ollama modelsoptimizing rag techniques based on locally deployed ollama modelsa case study with locally deployed ollama models. In *Proceedings of the 2024 3rd International Conference on Artificial Intelligence and Intelligent Information Processing*, AIIIP '24, page 152–159, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400707308. doi: 10.1145/3707292.3707358. URL https://doi.org/10.1145/3707292.3707358.

[46] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12: 157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

[47] Tie-Yan Liu. Learning to rank for information retrieval. *Found. Trends Inf. Retr.*, 3(3):225–331, March 2009. ISSN 1554-0669. doi: 10.1561/1500000016. URL https://doi.org/10.1561/1500000016.

[48] Carlos Llopis-Albert, Francisco Rubio, and Francisco Valero. Impact of digital transformation on the automotive industry. *Technological Forecasting and Social Change*, 162:120343, 2021. ISSN 0040-1625. doi: https://doi.org/10.1016/j.techfore.2020.120343. URL https://www.sciencedirect.com/science/article/pii/S0040162520311690.

[49] Henry Lopez-Vega et al. Digital transformation of the automotive industry. *International Journal of Operations & Production Management*, 2023. doi: 10.1080/13662716.2022.2151873.

[50] Michael Lystbæk and {Ulrik Sahl} Lystbæk. A norm-chatbot: Local llm vision with vision-based rag for complex production documents and task-specific responses. In Lazaros Iliadis, Ilias Maglogiannis, Efthyvoulos Kyriacou, and Chrisina Jayne, editors, *Engineering Applications of Neural Networks - 26th International Conference, EANN 2025, Proceedings*, pages 17–30, Netherlands, July 2025. Springer. ISBN 978-3-031-96198-4. doi: 10.1007/978-3-031-96199-1_2. 26th International Conference, EANN 2025 ; Conference date: 26-06-2025 Through 29-06-2025.

[51] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(4):824–836, April 2020. ISSN 0162-8828.

doi: 10.1109/TPAMI.2018.2889473. URL https://doi.org/10.1109/TPAMI.2018.2889473.

[52] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. Docvqa: A dataset for vqa on document images, 2021. URL https://arxiv.org/abs/2007.00398.

[53] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey, 2025. URL https://arxiv.org/abs/2402.06196.

[54] Sunil Nair, Jose Luis de la Vara, Mehrdad Sabetzadeh, and Lionel Briand. An extended systematic literature review on provision of evidence for safety certification. *Information and Software Technology*, 56(7):689–717, 2014. ISSN 0950-5849. doi: https://doi.org/10.1016/j.infsof.2014.03.001. URL https://www.sciencedirect.com/science/article/pii/S0950584914000603.

[55] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert, 2020. URL https://arxiv.org/abs/1901.04085.

[56] Linke Ouyang, Yuan Qu, Hongbin Zhou, Jiawei Zhu, Rui Zhang, Qunshu Lin, Bin Wang, Zhiyuan Zhao, Man Jiang, Xiaomeng Zhao, Jin Shi, Fan Wu, Pei Chu, Minghao Liu, Zhenxiang Li, Chao Xu, Bo Zhang, Botian Shi, Zhongying Tu, and Conghui He. Omnidocbench: Benchmarking diverse pdf document parsing with comprehensive annotations, 2025. URL https://arxiv.org/abs/2412.07626.

[57] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.

[58] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. KILT: a benchmark for knowledge intensive language tasks. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard,

Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2523–2544, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.200. URL https://aclanthology.org/2021.naacl-main.200/.

[59] Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. Doclaynet: A large human-annotated dataset for document-layout segmentation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, page 3743–3751, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393850. doi: 10.1145/3534678.3539043. URL https://doi.org/10.1145/3534678.3539043.

[60] Peng Qi, Xiaowen Lin, Leo Mehr, Zijian Wang, and Christopher D. Manning. Answering complex open-domain questions through iterative query generation. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2590–2602, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1261. URL https://aclanthology.org/D19-1261/.

[61] Renyi Qu, Ruixuan Tu, and Forrest Sheng Bao. Is semantic chunking worth the computational cost? In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2155–2177, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.114. URL https://aclanthology.org/2025.findings-naacl.114/.

[62] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. RocketQA: An optimized training approach to dense passage retrieval for open-domain question answering. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5835–5847, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.466. URL https://aclanthology.org/2021.naacl-main.466/.

[63] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, December 2023. doi: 10.1162/coli_a_00486. URL https://aclanthology.org/2023.cl-4.2/.

[64] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3(4):333–389, April 2009. ISSN 1554-0669. doi: 10.1561/1500000019. URL https://doi.org/10.1561/1500000019.

[65] Jon Saad-Falcon, Joe Barrow, Alexa Siu, Ani Nenkova, Seunghyun Yoon, Ryan A. Rossi, and Franck Dernoncourt. PDFTriage: Question answering over long, structured documents. In Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 153–169, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.13. URL https://aclanthology.org/2024.emnlp-industry.13/.

[66] Pranab Sahoo, Prabhash Meharia, Akash Ghosh, Sriparna Saha, Vinija Jain, and Aman Chadha. A comprehensive survey of hallucination in large language, image, video and audio foundation models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 11709–11724, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.685. URL https://aclanthology.org/2024.findings-emnlp.685/.

[67] Ahmmad O. M. Saleh, Gokhan Tur, and Yucel Saygin. SG-RAG: Multi-hop question answering with large language models through knowledge graphs. In Mourad Abbas and Abed Alhakim Freihat, editors, *Proceedings of the 7th International Conference on Natural Language and Speech Processing (ICNLSP 2024)*, pages 439–448, Trento, October 2024. Association for Computational Linguistics. URL https://aclanthology.org/2024.icnlsp-1.45/.

[68] Christopher Sciavolino, Zexuan Zhong, Jinhyuk Lee, and Danqi Chen. Simple entity-centric questions challenge dense retrievers. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6138–6148, Online and Punta Cana, Dominican Republic, November 2021. As-

sociation for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.496. URL https://aclanthology.org/2021.emnlp-main.496/.

[69] Lei Sheng and Shuai-Shuai Xu. Pdftable: A unified toolkit for deep learning-based table extraction. *ArXiv*, abs/2409.05125, 2024. URL https://api.semanticscholar.org/CorpusID:272524936.

[70] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

[71] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.

[72] Nicola Stokes, Yi Li, Lawrence Cavedon, and Justin Zobel. Exploring abbreviation expansion for genomic information retrieval. In Nathalie Colineau and Mark Dras, editors, *Proceedings of the Australasian Language Technology Workshop, ALTA 2007, Melbourne, Australia, December 10-11, 2007*, pages 100–108. Australasian Language Technology Association, 2007. URL https://aclanthology.org/U07-1015/.

[73] Alessandro Stolfo. Groundedness in retrieval-augmented long-form generation: An empirical study. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1537–1552, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-naacl.100. URL https://aclanthology.org/2024.findings-naacl.100/.

[74] Dewang Sultania, Zhaoyu Lu, Twisha Naik, Franck Dernoncourt, David Seunghyun Yoon, Sanat Sharma, Trung Bui, Ashok Gupta, Tushar Vatsa, Suhas Suresha, Ishita Verma, Vibha Belavadi, Cheng Chen, and Michael Friedrich. Domain-specific question answering with hybrid search, 2024. URL https://arxiv.org/abs/2412.03736.

[75] Haitian Sun, Tania Bedrax-Weiss, and William Cohen. PullNet: Open domain question answering with iterative retrieval on knowledge bases and text. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and*

the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 2380–2390, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1242. URL https://aclanthology.org/D19-1242/.

[76] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/65b9eea6e1cc6bb9f0cd2a47751a186f-Paper-round2.pdf.

[77] Vishesh Tripathi, Tanmay Odapally, Indraneel Das, Uday Allu, and Biddwan Ahmed. Vision-guided chunking is all you need: Enhancing rag with multimodal document understanding, 2025. URL https://arxiv.org/abs/2506.16035.

[78] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10014–10037, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.557. URL https://aclanthology.org/2023.acl-long.557/.

[79] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

[80] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.

[81] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. ISSN 00994987. URL http://www.jstor.org/stable/3001968.

[82] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, and Chun Jason Xue. Retrieval-augmented generation for natural language processing: A survey, 2025. URL https://arxiv.org/abs/2407.13193.

[83] Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. Layoutlm: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '20, page 1192–1200, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450379984. doi: 10.1145/3394486.3403172. URL https://doi.org/10.1145/3394486.3403172.

[84] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models, 2025. URL https://arxiv.org/abs/2401.11817.

[85] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation, 2024. URL https://openreview.net/forum?id=JnWJbrnaUE.

[86] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL https://aclanthology.org/D18-1259/.

[87] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. 2023. Publisher Copyright: © 2023 11th International Conference on Learning Representations, ICLR 2023. All rights reserved.; 11th International Conference on Learning Representations, ICLR 2023 ; Conference date: 01-05-2023 Through 05-05-2023.

[88] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. TaBERT: Pretraining for joint understanding of textual and tabular data. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages

8413–8426, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.745. URL https://aclanthology.org/2020.acl-main.745/.

[89] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. *Evaluation of Retrieval-Augmented Generation: A Survey*, page 102–120. Springer Nature Singapore, 2025. ISBN 9789819610242. doi: 10.1007/978-981-96-1024-2_8. URL http://dx.doi.org/10.1007/978-981-96-1024-2_8.

[90] Qintong Zhang, Bin Wang, Victor Shea-Jay Huang, Junyuan Zhang, Zhengren Wang, Hao Liang, Conghui He, and Wentao Zhang. Document parsing unveiled: Techniques, challenges, and prospects for structured information extraction. 2024. URL https://api.semanticscholar.org/CorpusID:273654101.

[91] Yunjia Zhang, Jordan Henkel, Avrilia Floratou, Joyce Cahoon, Shaleen Deep, and Jignesh M. Patel. Reactable: Enhancing react for table question answering. *Proc. VLDB Endow.*, 17(8):1981–1994, April 2024. ISSN 2150-8097. doi: 10.14778/3659437.3659452. URL https://doi.org/10.14778/3659437.3659452.

[92] Shuo Zhao and Xin Sun. Enabling controllable table-to-text generation via prompting large language models with guided planning. *Knowledge-Based Systems*, 304:112571, 2024. ISSN 0950-7051. doi: https://doi.org/10.1016/j.knosys.2024.112571. URL https://www.sciencedirect.com/science/article/pii/S095070512401205X.

[93] Yiyun Zhao, Prateek Singh, Hanoz Bhathena, Bernardo Ramos, Aviral Joshi, Swaroop Gadiyaram, and Saket Sharma. Optimizing LLM based retrieval augmented generation pipelines in the financial domain. In Yi Yang, Aida Davani, Avi Sil, and Anoop Kumar, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 279–294, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-industry.23. URL https://aclanthology.org/2024.naacl-industry.23/.

[94] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Proceedings of the 37th International Conference on Neural*

Information Processing Systems, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.

[95] Zijie Zhong, Hanwen Liu, Xiaoya Cui, Xiaofan Zhang, and Zengchang Qin. Mix-of-granularity: Optimize the chunking granularity for retrieval-augmented generation. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5756–5774, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL https://aclanthology.org/2025.coling-main.384/.

[96] Miao Zhu and Jacqueline Cole. Pdfdataextractor: A tool for reading scientific text and interpreting metadata from the typeset literature in the portable document format. *Journal of Chemical Information and Modeling*, 62, 03 2022. doi: 10.1021/acs.jcim.1c01198.

[97] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. Large language models for information retrieval: A survey. *ACM Trans. Inf. Syst.*, 44(1), November 2025. ISSN 1046-8188. doi: 10.1145/3748304. URL https://doi.org/10.1145/3748304.

Appendix

A

A.1 Dataset Links

- 100-QA
  `https://github.com/Shubhangi134/master-thesis/blob/main/raw_data/dataset/Questions_Answer.xlsx`
- FRAMES
  `https://huggingface.co/datasets/google/frames-benchmark`

A.2 Code

- Github Repository
  `https://github.com/Shubhangi134/master-thesis`

A.3 Prompt Templates

This section lists the prompt templates used for answer generation in the experiments. The same system prompt is used across settings, while the user prompt differs between the PDF dataset and the FRAMES dataset.

A.3.1 System Prompt

```
You are a helpful assistant.
```

A.3.2 PDF Dataset Prompt

```
You are an expert in automotive safety standards.
Answer the question in brief based ONLY on the following context.
Do not give explanations or additional information.
Output is expected to be in few words and not sentences.
If multiple answers are possible, separate them by semicolons or & symbol.
If the answer is not in the context, say "I do not know".

Context:
{context}

Question: {question}
```

A.3.3 FRAMES Dataset Prompt

```
Answer the question based ONLY on the provided context.
Keep it short and factual. If not answerable from context, say "I don't know".

Context:
{context}

Question: {question}
```

A.3.4 Multi-hop Judge Prompt

In the multi-hop setting, a separate prompt is used to decide whether the currently retrieved evidence is sufficient to answer the question.

Judge System Prompt

```
You are a strict judge for a retrieval-augmented QA system.
Answer YES only if the question can be
answered using verbatim content in the excerpts.
If any required numbers, IDs, clauses,
entities, or definitions are missing in the excerpts, answer NO.
Return ONLY valid JSON.
```

Judge User Prompt

```
Return JSON with keys:
  "answerable": "YES" or "NO"
  "missing": list of short labels
  "reason": <=25 words

Question:
{question}

Excerpts:
{evidence}
```